Capstone

Team

2026-04-24

Table of contents

Introduction

1 Proposal

Design and Pricing of a Fishable Days Index Derivative Incorporating Weather Risk and Regulatory Closures Using a Regime-Switching Model

Proposal Submitted Feb 2, 2026

1.1 Introduction and Objective

Fishermen often receive low income due to other reason beyond their control. Due to weather and ocean conditions, it will not be safe to fish. Additionally, fisheries are sometimes closed deliberately to allow marine ecosystems to recover, protect breeding times, and allow stocks to replenish. While environmentally necessary, these restrictions diminish short-term fishing opportunities and income. The aim of this project is to create and cost a parametric derivative agreement that provides financial protection when fishing days are lost due to negative environmental situation and regulatory closures. The Fishable Days Index (FDI) contract counts the days in a season when fishing is legally permitted and environmentally friendly. This project creates a quantitative framework with three objectives: (1) defines the fishable days using weather, ocean, and regulatory information, (2) models the stochastic behavior of fishable and unfishable days through a regime-switching approach, and (3) prices an index-based derivative by Monte Carlo simulation. The system is based on weather derivative pricing but altered slightly to fit the operational and regulatory realities of fisheries.

1.2 Why This Project Is Necessary

Income shocks have a big impact on small fisheries. The performance of traditional indemnity insurance in this sector is not up to the mark as it becomes difficult to verify losses and claims are settled slowly and controlling moral hazard is difficult. For this reason, many fishers are not insured. Parametric and index-based products provide clear and low-cost alternatives, as payouts depend on objective indices rather than observed losses (Skees, 2008). Apart from this, fisheries management depends on environmental regulations. Fish breeding closures are necessary to ensure sufficient aquatic animal breeding in nature during specific periods. These

closures are expected and desired, but they still reduce short-term fishing income. Regulatory dimensions are rarely considered in current financial risk tools.

1.3 Fishable Days Index with Environmental and Regulatory Constraints

The Fishable Days Index is the core of the project.

1.4 Derivative Contract Design

The project will consider a put-style derivative written on the Fishable Days Index. The contract provides compensation when fishable days fall below the expected level (Turvey, 2001).

1.5 Model Choice: Regime-Switching with Regulatory Closures

This project will use a regime-switching model, also known as a Hidden Markov Model, to describe environmental conditions (Hardy, 2001; Frühwirth-Schnatter, 2006). This model will be combined with deterministic regulatory closures.

1.6 Data and Implementation Plan

Data on wind and waves have been taken daily and are available in the public domain such as that of NOAA. Regulatory closure calendars will be sourced from fishery management agencies. The Fishable Days Index will be validated using fishing activity data, when available.

1.7 Expected Contribution

This project would develop a new index-based risk management framework for environmental protection policies. It aids in creating transparent and fair parametric goods for small-scale fisheries.

2 Fisher Rainy Day Fund – A Parametric Insurance Policy

MAP 5629: Quantitative Capstone in Environmental Finance

2026-04-24

Dr. Pablo Olivares

Fisher Rainy Day Fund – A Parametric Insurance Policy

2.1 Introduction

Fishermen are often unable to produce income due to reasons beyond their control. For example, weather and ocean conditions can make it unsafe to go fishing. Additionally, fisheries are sometimes closed deliberately to protect spawning times, ensure sustainable harvest rates, and allow overexploited stocks to replenish. While environmentally necessary, these restrictions diminish short-term fishing opportunities and income. The aim of this project is to create and cost a parametric derivative agreement that provides financial protection when fishing days are lost due to objectively defined negative environmental conditions.

The proposed Fisher Rainy Day Fund is a parametric insurance policy that is based on a Fishable Days Index (FDI). The contract counts the days in a season when fishing is legally permitted and environmentally feasible. This project uses a quantitative framework with three objectives: (1) define the fishable days using wind and wave meteorological conditions, (2) model the stochastic behavior of fishable and unfishable days through a Poisson-binomial distribution, and (3) price an index-based derivative. The system is based on weather derivative pricing but altered slightly to fit the operational realities of a small-scale island-based fisheries.

2.2 Background

Income shocks have a big impact on small fisheries. The performance of traditional indemnity insurance in this sector is not up to the mark as it becomes difficult to verify losses and claims are settled slowly. Furthermore, controlling moral hazard is difficult. For this reason, many fishers are not insured. Parametric and index-based products provide clear and low-cost alternatives, as payouts depend on objective indices rather than observed losses (Skees 2008).

2.3 Methods

2.3.1 Sources of Data

This study integrates four primary datasets to construct and validate the Fishable Days Index (FDI) for the St. Croix commercial fishery that targets Caribbean Spiny Lobster. Each dataset spans a study period from January 1, 1985, to December 28, 2024, resulting in a daily time series of 14,607 observations. All data loading, merging, and transformation were performed with the programming language "Python" to ensure full reproducibility and to avoid manual manipulation of the four source files.

The first dataset consists of daily commercial catch reports from individual fishers (Center 2026). Each observation records information for every fishing trip made by vessels fishing in St. Croix. The important variables utilized for this analysis were the date, total number of trips, number of trips specifically for the Caribbean Spiny Lobster (CSL), and total pounds of all species caught alongside pounds of solely CSL. The dataset is a limited event log: rows are present only on days when at least three fishers filed a self-reported logbook. Days with fewer than three individual reports are suppressed to zero to protect fisher privacy, meaning that missing continuous-date rows represent a mix of truly inactive days and days with only one or two reported trips.

The second and third datasets of wind speed and wave height, respectively, are derived from the ERA5 reanalysis product produced by the European Centre for Medium-Range Weather Forecasts (ECMWF). ERA5 provides hourly estimates of atmospheric and oceanic variables at a grid resolution of 31 km from 1940 onwards, reconstructed from historical observations and numerical weather prediction models (Hersbach et al. 2020). The wind dataset provides hourly eastward, u10, and northward, v10, wind components from 10 meters above the surface, as well as the instantaneous wind gust speed, fg10, for a single grid point representative of the St. Croix region (17.75°N, 64.75°W). The wave dataset provides hourly significant wave height, swh, at a nearby grid point (18.0°N, 64.5°W), defined as the average height of the greatest one-third of waves.

The fourth dataset consists of annual total population estimates for the United States Virgin Islands sourced from the World Bank, series POPTOTVIA647NWDB. Population data are used solely to normalize fishing activity metrics and remove secular trends in fleet capacity from the validation analysis, as described further below.

2.3.1.1 Daily Calendar Framework Construction

A complete daily calendar skeleton was constructed spanning the full study period using pandas date_range functionality, producing a main data frame of explicitly 14,607 rows. The limited fishing report file was left-joined onto this skeleton so that days absent from the original file became explicit rows with zero values for all fishing activity columns rather than structural absences. This distinction is significant: a day with zero reported trips carries different analytical meaning depending on whether it reflects true fishing trip inactivity, privacy-based censoring of low-report days, or adverse weather conditions. The calendar join method makes all three types of zero-day observable within the same framework.

Hourly ERA5 wind data were processed by first computing the scalar wind speed from the u and v vector components as the Euclidean magnitude, expressed formally as $|W| = \sqrt{u_{10}^2 + v_{10}^2}$ with units of meters per second (m/s). Wind speeds were additionally converted to knots using the exact conversion factor of $1 m/s = 1.9438449$ knots. Both scalar wind speed and gust speed were then transformed from hourly to daily frequency by taking the maximum of the daily values. The choice of daily maximum rather than daily mean reflects the operational reality that a single period of dangerous wind or wave conditions within a day is sufficient to prevent fishing activity regardless of conditions during other hours. This process was replicated to transform hourly significant wave heights to daily maximum values.

To isolate climate-driven variation in lobster fishing activity from background trends driven by changes in the island's population and fishing fleet capacity, all fishing activity metrics used in the threshold validation were normalized by annual population. Specifically, CSL trip rates were expressed as the number of lobster trips per 1,000 residents, and CSL catch rates were expressed as pounds of lobster caught per 1,000 residents per day. This normalization process allows for using the activity rates of fishing rather than raw counts, ensuring that this signal reflects responses to environmental conditions rather than changes driven by population demands.

The four datasets were merged into a single main daily table using calendar date as the primary join factor for climate data and integer year as the join factor for annual population values. Forward-filling was applied to any missing climate values arising from gaps in hourly ERA5 coverage, carrying the most recent available observation forward. The final main table contains 14 columns covering key information of date identifiers, fishing activity metrics, daily maximum wind speed and gust in both m/s and knots, daily maximum significant wave height, and annual population.

2.3.1.2 Weekday Behavioral Bias

Prior to threshold calibration, a weekday bias analysis was conducted to determine whether fishing activity exhibited systematic day-of-week patterns unrelated to climate conditions. Mean population-adjusted lobster trip rates and the proportion of days with any reported lobster activity were computed for each day of the week across the full 40-year study period.

The analysis revealed a pronounced decline in fishing activity on Sunday relative to all other days of the week. The mean CSL trip rate for just Sundays had a value of 0.016 trips per 1,000 people compared to a range of 0.034 to 0.046 on all other weekdays, and a day-active rate of 32.1% compared to 54.4% to 62.8% on other days. This pattern is consistent with traditions of the United States Virgin Islands, where a decrease in labor on Sunday is culturally normative and not a response to environmental conditions. Sunday observations were therefore excluded from the climate threshold validation analysis to prevent behavioral suppression from contaminating the climate signal. Sunday observations were retained within the Fishable Days Index construction itself, as the index measures climate-based fishability rather than observed fishing behavior.

2.3.1.3 FDI Definition and Threshold Selection

The Fishable Days Index (FDI) is defined as a binary daily indicator equal to one ($FDI = 1$ on safe days) on days when environmental conditions are consistent with safe small-vessel lobster fishing operations, otherwise FDI is equal to zero ($FDI = 0$ on unsafe days). For the St. Croix CSL fishery, the index is determined by two simultaneous climate conditions: daily maximum wind speed must not exceed a specified limit, and daily maximum significant wave height must not exceed a separate specified limit. Since lobster fishing in St. Croix is open year-round with no regulatory seasonal closure, the FDI contains no regulatory component and is determined entirely by the environmental conditions. Therefore, the annual FDI for any given season is the sum of daily FDI values over a 365-day calendar year.

Initial threshold values were drawn from the small vessel operability literature. Wind speeds at or above 20 knots (approximately 10.3 m/s) are widely cited as the threshold above which small craft advisories apply in United States coastal waters, and significant wave heights above 2.0 to 2.5 meters are considered the upper operational limit for vessels typically used in St. Croix lobster fisheries. These values were used initially as starting parameters and evaluated against observed fishing activity data.

2.3.1.4 Threshold Validation

The selected thresholds were validated by testing whether days classified as fishable demonstrated higher population-adjusted lobster fishing activity than days classified as unfishable

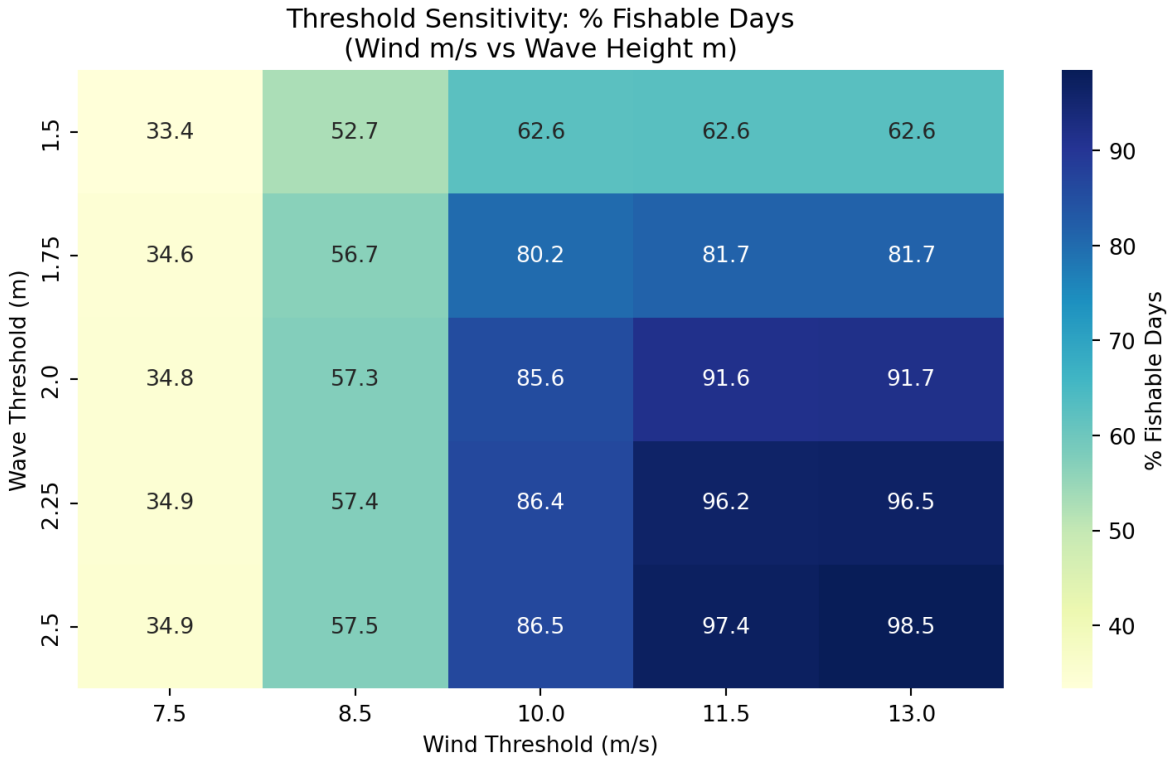


Figure 2.1: Heatmap of threshold sensitivity analysis for varying fishable day weather conditions

and validated with the fishing report data. Sunday observations were excluded from this analysis as described previously in Section 2.3.1.2. This validation compared mean CSL trip rates per 1,000 people and the proportion of days with any reported lobster activity between the two FDI categories.

Using the final thresholds (8.5 m/s and 1.75 m), fishable days had an average CSL trip rate of 0.044 trips per 1,000 people compared to 0.039 on unfishable days, and an active-day rate of 62.5% in contrast to 56.8% on unfishable days. This difference is understood to be conservative relative to what the true climate signal likely produces. The privacy-based censoring rule applied to the fishing report data suppresses observations to zero on any day with fewer than three individual fisher reports. This censoring feature disproportionately affects days near the threshold boundary, where marginal fishing conditions may motivate only one or two fishers to go out, which are the most informative observations for threshold calibration. Therefore, the true separation between fishable and unfishable days is likely greater than these validation statistics suggest.

The sensitivity of the validation result to the threshold values was additionally assessed by identifying that wind speed is the binding constraint in the St. Croix context. Inspection of the sensitivity heatmap reveals that varying the wave threshold from 1.5 to 2.5 meters at a fixed wind threshold of 8.5 m/s changes the fishable day rate by only 4.8 percentage points (52.7% to 57.5%), while varying the wind threshold from 7.5 to 13.0 m/s at a fixed wave height of 1.75 meters produces a range from 34.6% to 81.7%.

2.3.2 Methodology for Pricing the Weather Derivative

2.3.2.1 Contract purpose and pricing logic

The agreement shifts the risk of adverse weather conditions from the derivative buyer to the seller. The contract will not track the actual day-to-day business loss. In contrast, it observes a weather-based index. The design is based on the essential structure of weather derivatives and index insurance. Buyer pays a premium upfront (Alaton, Djehiche, and Stillberger 2002; Cao and Wei 2004). If the index exceeds the trigger threshold during the contract period, the seller makes the contract payment. The contract reduces loss-adjustment costs by generating objective settlements from the recorded weather data and a priori estimated losses instead of from audited firm losses (Bemš and Aydin 2021; World Bank 2023).

The contract value in this report is based on an actuarial expected payoff. Since the underlying weather variable is not traded in deep or futures markets, the choice of a predetermined payoff fits well with weather derivatives. Earlier research applies identical expected loss analysis reasoning for temperature, rainfall, and agricultural weather-index contracts (Alaton, Djehiche, and Stillberger 2002; Bemš and Aydin 2021; Richards, Manfredo, and Sanders 2004). Some other pricing models can be mentioned as well, e.g. equilibrium, indifference, or copula-based approaches. However, the actuarial approach provides a clear starting point and aligns with

available data and the objective of pricing a fishability-based trigger from past weather conditions (Bressan and Romagnoli 2021; Cao and Wei 2004; Xu, Odening, and Musshoff 2008).

2.3.2.2 Index definition and contract trigger

Let the contract cover T calendar days. For each day j, define an indicator variable X_j. Set $X_j = 1$ when observed weather conditions support fishing on day j. Set $X_j = 0$ when weather conditions do not support fishing. The total number of fishable days will be counted by the contract over the entire window. Through this index, a noisy weather sequence is reduced to one contract level quantity that is easily priced by the seller and interpretable by the buyer. The cited framework is consonant with standard weather-derivative design, whereby pay-off is based on a transparent index rather than on realized revenue or profit (Bemš and Aydin 2021; Jewson, Brix, and Ziehmann 2005; World Bank 2023).

The seasonal fishable-day index is $N = \sum_{j=1}^{n} X_i$. Here, N counts the number of fishable days in the contract period. Let K denote the strike on fishable days. The contract triggers when fishable days fall below the strike. In mathematical form, the trigger event equals $N < K$. That event defines the "limiting" or threshold-crossing event for the contract. A higher strike makes the trigger easier to hit. A lower strike makes the trigger harder to hit (World Bank 2023).

2.3.2.3 Historical estimation of daily fishability probabilities

For every contract day, we estimate the fishable-day probability. Let p_j represent the probability that day j is fishable. The p_j estimates take place after the index classifies each historical day as either fishable or non-fishable. In effect, we compute one empirical probability for every calendar day in the contract window, pooling the same month-day across historical years. That measure saves seasonality. In addition, it gives January and September distinct probabilities. This step is important because weather derivatives usually require careful treatment of seasonality and calendar structure prior to pricing (Alaton, Djehiche, and Stillberger 2002; Jewson, Brix, and Ziehmann 2005).

Instead of imposing a single probability common to every day, we allow every day to have its own estimated probability p_j. This flexibility inspires the Poisson-binomial model. All daily probabilities would have to be matched by the ordinary binomial model. The Poisson-binomial model can handle independent daily Bernoulli trials with unequal success probabilities, which fits a seasonal weather calendar much better (Biscarri, Zhao, and Brunner 2018; Chen and Liu 1997; Hong 2013).

2.3.3 Poisson-Binomial Model

2.3.3.1 Why this model fits the problem

The Poisson-binomial distribution signifies the total of independent Bernoulli variables that need not share the same success probability. It harmonizes with the contractual weather-based index in a natural fashion as there are predefined conditions that define total fishable days. Every day has its own historical fishability probability p_j. After considering the daily indicators, the model considered daily probabilities as independent and treats them as Bernoulli variables (Fishable or Non-fishable). The seasonal count N was made to follow a Poisson-binomial distribution. Research in statistics on this distribution has established the theory, computation and use of a distribution with a trial probability that varies (Chen and Liu 1997; Hong 2013). Formally, $X_j \sim \text{Bernoulli}(p_j), \quad j = 1, 2, \dots, T$ and assume independence across days, $N = \sum_{j=1}^{T} X_j \sim \text{Poisson-Binomial}(p_1, p_2, \dots, p_T)$. The expected number of fishable days follows, $E[N] = \sum_{j=1}^{T} p_j$. The variance follows, $Var(N) = \sum_{j=1}^{T}(1 - p_j)$.These moments summarize the center and spread of the seasonal fishable-day distribution under the independence assumption.

2.3.3.2 Probability of mass function

The exact probability that the season ends with exactly fishable days equals

$$P(N = n) = \sum_{A \epsilon F_n} \prod_{j \epsilon A p_j} (1 - p_j), \quad n = 1, 2, \dots, T$$

Where F_n denotes the collection of all subsets of $\{1, \dots, T\}$ that contain exactly n elements. This equation sums up the probability of every possible path that produces exactly n fishable days. The equation gives the full probability mass function of the contract index. It also provides the foundation for trigger probability and price calculations (Chen and Liu 1997; Fernandez and Williams 2010; Hong 2013).

This report does not enumerate all subsets directly because that route grows expensive when T becomes large. Instead, the code uses recursive convolution, which updates the distribution one day at a time. Let $f_m(n)$ denote the probability of obtaining n fishable days after the first m contract days. Start with $f_0(0) = 1, f_0(n) = 0, \text{ for } n > 0.$

Then update with $f_m(n) = (1 - p_m) f_{m-1}(n) + p_m f_{m-1}(n - 1), \text{ for } m = 1, 2, \dots, T$ for all valid n. After the last update, $f_T(n) = P(N = n)$. Statistical studies show that recursive and Fourier-based methods provide exact or highly efficient ways to compute the Poisson-binomial distribution in applied work (Biscarri, Zhao, and Brunner 2018; Chen and Liu 1997; Fernandez and Williams 2010; Hong 2013).

2.3.4 Limiting Probability Equation

2.3.4.1 Trigger probability under the strike limit

The key risk quantity in this contract comes from the probability that fishable days will fall below the strike. That probability drives the contract price directly since the payoff uses a fixed indemnity. Let $\Psi(K)$ denote the limiting probability of the contract trigger.

Then $\Psi(K) = P(N < K) = \sum_{n=0}^{K-1} P(N = n)$. This equation gives the cumulative lower-tail probability of the Poisson-binomial distribution up to one day below the number of days associated with the strike percentage times the mean fishable days. It answers the central pricing question: how often would the season fail to reach the minimum acceptable number of fishable days? Hong (2013). If the strike comes from a percentage of expected fishable days, the model can define $K_s = round(s\mu), \mu = E[N] = \sum_{j=1}^{T} p_j$

where the value of strike percentage is explored from 0.80 to 0.99 The strike percentage makes the conversion into an integer number of fishable days. The model then calculates $\Psi(K)$ for each level of strike. This threshold design is consistent with common weather index practice where the contract maps an index threshold into a trigger probability and then to an expected payout.

2.3.4.2 Payoff Design

2.3.4.3 General payoff function

A weather derivative price starts with the payoff function. Let $G(N)$ denote the contract payoff at expiration as a function of the seasonal fishable-day count N. The most general actuarial pricing equation then discounts the expected payoff back to the pricing date:

$$V_0 = e^{-r\tau} E[G(N)] = e^{-r\tau} \sum_{n=0}^{T} G(n) P(N = n)$$

Where r denotes the continuously compounded risk-free rate and τ denotes the contract length in years. This expected-payoff structure sits at the center of actuarial and burn-analysis pricing for weather-index products (Alaton, Djehiche, and Stillberger 2002; Jewson, Brix, and Ziehmann 2005; Taib and Benth 2012; World Bank 2023).

2.3.4.4 Fixed-payout contract used in the code

We use a predetermined and fixed payout. The payout would be issued if the total number of days in the contract period falls below the strike. Let L denote that payout. Then the payoff function equals $G(N) = L1(N < K)$ where $1(N < K)$ equals 1 when the trigger hits, and 0 otherwise.

Under this design, the weather derivative price simplifies to $V_0 = e^{-r\tau}LP(N < K) = e^{-r\tau}L\Psi K$. This form makes the pricing logic very clear. The discount factor handles time value. The limiting probability handles event likelihood. The payout level handles contract severity. Multiply the three terms, and the model returns the fair actuarial price (Jewson, Brix, and Ziehmann 2005; Taib and Benth 2012; World Bank 2023).

In implementation, the code sets the fixed payout from the shortfall between the expected fishable-day count and the strike, multiplied by average profit per day as $L = (\mu - K)\pi^-$, for $K \leq \mu$. Where π^- denotes average profit per fishable day. If the analyst wants a payout rate α, then the formula becomes $L = \alpha(\mu - K)\pi^-$. This rule creates a simple and transparent contract. It does not reward larger realized shortfalls once the trigger fires. It only pays one fixed amount per strike. That feature keeps pricing simple, but it also sacrifices some loss sensitivity (Kath et al. 2018; World Bank 2011, 2023).

2.3.4.5 Step-by-Step Pricing Procedure

2.3.4.6 Build the daily index

The meteorological department begins recording daily weather observations. The analyst selects weather thresholds that define what is deemed a fishable day. The wind speed and wave height provide thresholds for our study. Our code assigns each historical day to a fishable or non-fishable label. This step generates the binary series necessary for Bernoulli modeling. The weather-derivative practice usually begins at this stage of index construction, since the derivative settles on the index, not the raw weather variables (Bemš and Aydin 2021; Jewson, Brix, and Ziehmann 2005; World Bank 2023).

2.3.4.7 Estimate Day-specific probabilities

Next, the analyst estimates p_j for each calendar day in the contract. The code eliminates February 29 so that the calendar will have 365 days only. This then pools the month-day across all years and calculates that date's historical fishable frequency. After this step there will be one probability for each contract day. The method allows different seasons to take values independently without forcing all days to value from the same distribution. The Poisson-binomial choice is justified by that feature (Chen and Liu 1997; Hong 2013; Jewson, Brix, and Ziehmann 2005).

2.3.4.8 Compute the seasonal distribution

Once p_j values have been estimated for the model, the full Poisson-binomial probability mass function for N is computed. Each recursive update starts on day 1 and folds one day into the distribution repeatedly. The model has a probability for each possible total fishable-day count from 0 to T. A full distribution conveys more information than just the means. It enables the analyst to calculate trigger probabilities in the lower tail, expected payoffs, and premium sensitivity across various strike levels (Biscarri, Zhao, and Brunner 2018; Fernandez and Williams 2010; Hong 2013).

2.3.4.9 Interpretation of the Price

This price represents the current worth of the expected contract payoff calculated using the historical probability model. The price omits transaction costs, profit loading, administrative charges, or a market price of weather risk. Research on weather derivatives indicates that incomplete-market settings can justify alternative pricing rules, including the use of equilibrium or indifference methods when traders demand compensation for non-hedgeable weather risk. However, our Poisson-binomial price based on actuarial technique is suitably transparent and represents a fair-value benchmark (Alaton, Djehiche, and Stillberger 2002; Cao and Wei 2004; Richards, Manfredo, and Sanders 2004; Xu, Odening, and Musshoff 2008).

As the strike percentage increases, the contract is triggered more easily because the season must meet a higher target for fishable days. Hence, the lower tail cumulative probability increases. Beneath a fixed payout design, the premium increases since the contract pays the same amount more often. With a shortfall design, the premium usually rises even faster as the contract pays out more often and can pay larger amounts. The upward pattern is what standard derivative and insurance intuition suggests: more coverage costs more (Kath et al. 2018; World Bank 2023).

2.3.4.10 Results and Findings

2.3.4.11 Threshold sensitivity and final index choice

The threshold sensitivity analysis demonstrates (Table 1) that the fishable-day index is quite responsive to the chosen wind and wave cutoffs. Reduced thresholds yield a significantly more limiting index, where most days are classified as non-fishable. A higher threshold was identified as more permissive (classifying most and almost all days as fishable). That methodology section states that the daily fishability indicator is determined by whether both the wind speed and the wave height do not exceed the cutoffs chosen. The sensitivity table indicates that for the strictest threshold pair, the percent of fishable days is around 33.35% (7.5 m/s, 1.5 m). However, for the loosest threshold pair, the percent of fishable days is around 98.47% (13.0 m/s, 2.5 m). The average yearly fishable day index moves in the same manner. It increases

from roughly 121.80 days to 359.60 days. The yearly deviation does not monotonically shift throughout all threshold pairs, but it does drop when the threshold becomes very loose and ends up with the index approaching the ceiling of the annual calendar, leaving less room for interannual fluctuation.

The pair of 8.50 m/s for wind and 1.75 m for wave height selected provides a balanced index (Table 1). This configuration results in 56.67% fishable days, a mean annual fishable day count of 206.95, and an annual standard deviation of 23.95 over the total historical record. This selection bypasses the two weak extremes. If the threshold is very strict, it will classify too many days as non-fishable and compress the commercial meaning of the index. A highly lenient threshold would deem nearly every day suitable for fishing, weakening the derivative trigger itself. The chosen threshold thus conserves enough non-fishable days to generate variation in risk, and nevertheless, incurs plausible operating conditions. The chosen approach has a methodological objective of developing a weather-based index that accounts for actual fishing access rather than merely the meteorological variation directly.

2.3.4.12 Contract window summary and annual fishable-day distribution

The contract is for the full calendar year, January 1 to December 31. This design gives a 365-day window of exposure that covers the entire seasonal cycle in marine weather. The probability summary reveals that the range over the contract period is likely 206.80 fishable days. According to the annual contract FDI summary (Table 2), the mean is 206.78 days with a standard deviation of 23.94 days. The two values are nearly equal. This close agreement suggests that the daily empirical probabilities correspond well to the historical annual counts. Essentially, the methodology's historical daily-probability construction produces the long-run average fishable day within a year with little error.

The 207 fishable days mean the selected threshold, supports fishing on roughly 57% of the days in a normal year (Table 2). An inter-annual variability of around 24 days is suggested by the standard deviation. This amount is relevant for pricing. A derivative contract must ensure sufficient dispersion in the index to create a meaningful lower- tail trigger probability. Here, the yearly difference facilitates the need. Simultaneously, the spread does not appear to be so large as to indicate an unstable index or poorly defined index. Consequently, this measure assesses fishable days and can be used as an underlying contract variable.

2.3.4.13 Seasonal pattern in daily fishability probability

The daily probability plot shows a clear seasonal pattern within the year. It follows from the methodology section that the model estimates probability separately for each contract day, pooling the same month-day over years. The curve in the plot does not remain flat. It demonstrates a strong seasonal structure instead. In the early season, the probabilities of being able to fish daily are around 0.35 to 0.50. Those probabilities rise during late winter and spring,

sharply dip around mid-contract year, and then rise again towards a second high-probability period later in the year (Figure 2).

This trend comes with two important repercussions. In the first place, the contract index does not follow a stationary Bernoulli process with a daily constant probability. The Poisson-binomial framework implemented in the methodology is justified by that result which allows unequal probability. Seasonal structure shows that risk is not spread evenly over the year, but has months dedicated to risk. A concentrated risk window with additional non-fishable days is marked by a mid-year dip. This period at the end of the year is under a high probability window. The seasonal variation influences the annual fishable days distribution of annual fishable days and the lower-tail trigger probability. The findings thus confirm that we were right to not homogenize all days into one average probability and preserve calendar-day heterogeneity.

According to the plot, it is observed that during the least favorable periods, many daily probabilities are well below 0.5, while during the most favorable periods, probabilities rise above 0.8. This range shows a strong seasonal contrast. The contrasting elements enhance the substance of the index. Furthermore, it upholds the pricing logic as seasonal structure contracts can capture real operational exposure better than those based on constant average hazard contracts.

2.3.4.14 Strike design and trigger behavior

The pricing analysis assesses strike levels between 80% and 95% of the expected average number of fishable days calculated over a one-year contract period. This design directly ties the available data to user-specified percentage of expected contract fishable days. The subsequent periods between strikes vary from 165 to 196 days (Table 3). With increasing strike percentage, the trigger threshold moves towards 207 days, which is the expected annual fishing days. This change raises the chances of payout due to increased low-event odds, but results in a lower payout since fewer days are factored into the losses.

The probabilities of triggers indicate a strong non-linearity response. The contract gets triggered rarely at lower levels. The prospect of a trigger only has a probability of 0.000002 at 80% strike. Further, it is 0.000004 at 81% and 0.000019 at 82%. These values suggest a nearly dormant contract. Even a very adverse weather year would have to fall well below the long-run mean before the payout condition trigger. As a result, the contract guarantees minimal practical protection in this strike range.

Once the strike crosses above the upper-80% level, the sensitivity of the trigger probability begins to move. It goes up to 0.001289 at 87%, 0.002605 at 88%, and 0.005032 at 89% (Table 3). The likelihood then quickly rises in the region of 90% to 95%. At 90%, it reaches 0.009303; at 91%, 0.016460; at 92%, 0.027895; at 93% it reaches 0.045302; at 94%, 0.070555; finally, at 95%, it reaches 0.105465. This pattern is non linear. The upper strike range penetrates further into the core of the annual fishable-day distribution. A slight adjustment in the strike rate, thus, has a larger impact on trigger frequency.

It holds practical significance. A designer who accepts strikes under 88% would probably be selling a product that has no economic relevance as the trigger probably never fires. An option that settles at the money or strikes near 93% to 95% will create a much more active hedging tool. Even at that range, an event-based contract would remain in place, no annual transfer. Consequently, the findings suggest that the upper strike range is the most defensible area for meaningful hedging.

2.3.4.15 Derivative price profile across strike levels

The continuously rising weather derivative price with respect to strike percentage faces a steep change with respect to strike. The fixed-payout pricing formula, $0 = -\ \Psi(\ )$, directly leads to this pattern. As the strike rises, the gap between the expected fishable days and the strike narrows, causing the fixed payout to decline in the current implementation. However, the payout falls faster than the trigger probability rises. This probability effect dominates the price function.

The price is practically nonexistent for low strikes. At 80% the amount is \$0.02, at 81% the amount is \$0.06, at 82% it is \$0.23 and at 83% it is \$0.54 (Table 3). Even at 86%, it costs under \$5.58. The values indicate that lower strike regions generate a very low expected protection value. The contract rarely activates to warrant a significant premium, according to the experts.

The price then increases quickly once the strike enters the low-90% range (Figure 3). It climbs to \$20.52 at 88%, \$36.45 at 89%, \$61.46 at 90%, \$98.28 at 91%, \$148.81 at 92%, \$212.86 at 93%, \$286.64 at 94%, \$361.38 at 95%. As a result, the price curve is convex. The derivative becomes markedly more costly as the strike nears the expected annual total fishable days.

This shape is economically viable. A higher strike position triggers nearer to the midpoint of the annual fishable-day distribution. While the fixed payout amount falls, the payout frequency increases with that move. It can be observed from the results that the rise in trigger probability surpasses the decrease in a benefit size (Figure 3). That collaboration leads to a sudden rise in the premium curve. As per the model, it is not the fixed payout size but rather triggers probability that drives price over the tested range.

2.3.4.16 Joint interpretation of trigger probability and premium

The trigger probability curve and the premium curve both show positive exponential patterns but with different rates. The difference is due to the pricing formula. The premiums reflect the low tail probability and the fixed payout level. In the design under test, the payout decreases as the strike percentage increases. However, the effect of lower tail probability has a stronger influence. The interaction generates a curve for the price per number of fishable days that initially rises smoothly and then increases at a faster rate (Figure 3 & Figure 4).

2.4 Discussion

This project did not explicitly consider related logistics and financial overheads of pricing a weather-based derivative to cover non-fishable days. Here, we discuss some of the logistics the proposed rainy day fisher fund would need to further consider including reinsurance, markups, and pooling risk. To offer the parametric weather-based insurance, the insurer would need to have sufficient funds and ensure that payouts would be rapid upon reaching strike conditions. Unless a fishing community were to develop a reciprocal insurance exchange, through a self-insurance pool with sufficient funds to cover potential payouts, the cost of issuing any parametric insurance policy would also need to include operating expenses and reinsurance. Since this analysis consisted only of a single industry located in a small geographic area, the proposed program could benefit from developing similar frameworks across a wider geographical region or including other types of industries to diversify risk. Such diversification would reduce the expenses of external reinsurance.

The process of pricing a parametric insurance policy serves as a simplified case study and exploration of the fundamental dynamics of pricing parametric insurance. First, we focused on developing a fishable day index as an objective metric evaluated from specific timeseries of meteorological data. Then we utilized pricing. Although we intentionally structured this analysis to be simple to interpret and evaluate, we acknowledge that refining the methods could lead to more accurate strike probabilities. High-quality fisher data and rationales for staying in port would be the primary tool for reducing basis risk. Direct conversations or surveys with fishers could improve the estimated accuracy of losses and help identify additional threshold conditions that allow effort adaptations versus conditions that impede all potential fishing activity.

The result of the sensitivity test showed that the index varied strongly with thresholds of winds and waves. The thresholds are too strict thus labeling too many days non-fishable, making the index less practical. Contract triggers become weaker when very loose thresholds classify all days as fishable. The 8.5 m/s wind speed and 1.75m wave height value were selected to avoid extremes. It creates about 206.95 fishable days each year and has enough annual variation for pricing. Recent studies connecting weather thresholds and sea-state limits to marine access and operational downtime are consistent with this outcome. As per research, what is selected as threshold must depict actual working conditions, and not just the meteorological variation (Chitteth Ramachandran et al. 2022; Costas et al. 2023; Karathanasi, Soukissian, and Hayes 2022). This threshold strikes a nice balance, as demonstrated in our study. It still preserves commercial meaning and reflects real fishing conditions.

The pricing results also strengthen the structure of contracts. The regular fishability likelihoods follow a clear seasonal pattern, so the risk does not remain constant throughout the year. The observed pattern is a strong endorsement for the implementation of the Poisson-binomial model which makes the probabilities daily contract period-varying. The analysis of strikes shows practically no protection from low strikes, i.e. strikes from 80% to 87%, which is hardly that the trigger will activate. On the other hand, 93% to 95% strikes lead to the tanks

producing a more active hedge with increased trigger probabilities and higher premiums. The evidence presented here is consistent with other studies on parametric insurance which find significant dependence of contract performance on trigger design and threshold placement which is typically at the core of the loss distribution (Larsson 2023; Xu, Odening, and Musshoff 2008). The data indicates that the upper strike range offers protection that is the most useful for fishers that want coverage during moderately adverse years, as opposed to only extreme years.

Currently, we still have limited information that we can use to independently determine fishable days. Although we did not consider that day-to-day business data would be required, information related to costs, revenue, and rationale for non-fished days would be critical to improve weather derivative pricing. Ideally, the benefits associated with rapid payments under a straightforward parametric insurance framework could serve as motivation for improving the quantity and quality of self-reported fisher activity. By establishing a transparent parametric framework, we were able to develop a user interface dashboard to communicate the weather derivative pricing over a range of strike percentages and different wind and wave threshold scenarios. By maintaining and improving this interactive dashboard, fishers would be able to customize the conditions to meet personal needs and preferences related to risk tolerance and budget. Ideally, creating a positive feedback loop of improved reporting, leading to more accurate thresholds, thereby reducing basis risk and making the insurance more affordable.

2.5 Conclusion

This study proposes the fishable-day index for developing parametric insurance contracts for fishers. The procedure creates a daily index based on wind speed and wave height. It then estimates day-specific probabilities and applies to a Poisson-binomial model for seasonality. The findings indicate that selection of thresholds strongly governs index behavior. Highly restrictive parameters significantly decrease fishing days. Loosening limitations removes risk. The chosen thresholds of 8.5 m/s for wind and 1.75 m for wave height create a balanced set of criteria. It generates approximately 207 fishable days annually and sustains enough variance for pricing. The validation confirms that fishable days correspond with a peak fishing activity, thus justifying the practical relevance of the index. The seasonal probability pattern bolsters the choice of model since it shows real changes in fishing conditions during the year, allowing us to avoid an assumption of constant risk. The pricing results have important economic consequences. Strike levels that are too low mean almost zero trigger probability, which also does not offer much protection. Increasing the strike levels increases the chances of triggering and the premium. The range from 93% to 95% offers the most suitable balance of cost and coverage. The contract can respond to conditions that are only moderate or worse rather than only extreme events. The convex price pattern indicates that pricing is driven more by trigger probability than by payout size. The paper also identifies major limitations. Operational costs, reinsurance, and market risk loading are not included. It is also dependent on limited data of fishing activity that may mask true behavior because of reporting limitations. In

the future, we must improve the quality of data, including fisher feedback, and further refine the threshold conditions. Broadening the framework to other regions or sectors would allow risk pooling. The research provides a well-defined and pragmatic framework for constructing a weather-based insurance instrument beneficial to fishers under uncertain conditions of the sea.

2.6 References

Alaton, Peter, Boualem Djehiche, and David Stillberger. 2002. "On Modelling and Pricing Weather Derivatives." *Applied Mathematical Finance* 9 (1): 1–20. https://doi.org/10.1080/13504860210132897.

Bemš, Július, and Caner Aydin. 2021. "Introduction to Weather Derivatives." *WIREs Energy and Environment* 11 (3). https://doi.org/10.1002/wene.426.

Biscarri, William, Sihai Dave Zhao, and Robert J. Brunner. 2018. "A Simple and Fast Method for Computing the Poisson Binomial Distribution Function." *Computational Statistics & Data Analysis* 122 (June): 92–100. https://doi.org/10.1016/j.csda.2018.01.007.

Bressan, Giacomo Maria, and Silvia Romagnoli. 2021. "Climate Risks and Weather Derivatives: A Copula-Based Pricing Model." *Journal of Financial Stability* 54 (June): 100877. https://doi.org/10.1016/j.jfs.2021.100877.

Cao, Melanie, and Jason Wei. 2004. "Weather Derivatives Valuation and Market Price of Weather Risk." *Journal of Futures Markets* 24 (11): 1065–89. https://doi.org/10.1002/fut.20122.

Center, Southeast Fisheries Science. 2026. "Caribbean ST Croix Logbook Survey (Vessels)." https://www.fisheries.noaa.gov/inport/item/30423.

Chen, Sean X., and Jun S. Liu. 1997. "Statistical Applications of the Poisson-Binomial and Conditional Bernoulli Distributions." *Statistica Sinica* 7 (4): 875–92. https://www.jstor.org/stable/24306160.

Chitteth Ramachandran, Rahul, Cian Desmond, Frances Judge, Jorrit-Jan Serraris, and Jimmy Murphy. 2022. "Floating Wind Turbines: Marine Operations Challenges and Opportunities." *Wind Energy Science* 7 (2): 903–24. https://doi.org/10.5194/wes-7-903-2022.

Costas, Raquel, Humberto Carro, Andrés Figuero, Enrique Peña, and José Sande. 2023. "A Decision-Making Tool for Port Operations Based on Downtime Risk and Met-Ocean Conditions Including Infragravity Wave Forecast." *Journal of Marine Science and Engineering* 11 (3): 536. https://doi.org/10.3390/jmse11030536.

Fernandez, Manuel, and Stuart Williams. 2010. "Closed-Form Expression for the Poisson-Binomial Probability Density Function." *IEEE Transactions on Aerospace and Electronic Systems* 46 (2): 803–17. https://doi.org/10.1109/taes.2010.5461658.

Hersbach, Hans, Bill Bell, Paul Berrisford, Shoji Hirahara, András Horányi, Joaquín Muñoz-Sabater, Julien Nicolas, et al. 2020. "The ERA5 Global Reanalysis." *Quarterly Journal of the Royal Meteorological Society* 146 (730): 1999–2049. https://doi.org/10.1002/qj.3803.

Hong, Yili. 2013. "On Computing the Distribution Function for the Poisson Binomial Distribution." *Computational Statistics & Data Analysis* 59 (March): 41–51. https://doi.org/10.1016/j.csda.2012.10.006.

Jewson, Stephen, Anders Brix, and Christine Ziehmann. 2005. "Weather Derivative Valuation," March. https://doi.org/10.1017/cbo9780511493348.

Karathanasi, Flora E., Takvor H. Soukissian, and Daniel R. Hayes. 2022. "Wave Analysis for Offshore Aquaculture Projects: A Case Study for the Eastern Mediterranean Sea." *Climate* 10 (1): 2. https://doi.org/10.3390/cli10010002.

Kath, Jarrod, Shahbaz Mushtaq, Ross Henry, Adewuyi Adeyinka, and Roger Stone. 2018. "Index Insurance Benefits Agricultural Producers Exposed to Excessive Rainfall Risk." *Weather and Climate Extremes* 22 (December): 1–9. https://doi.org/10.1016/j.wace.2018.10.003.

Larsson, Karl. 2023. "Parametric Heat Wave Insurance." *Journal of Commodity Markets* 31 (September): 100345. https://doi.org/10.1016/j.jcomm.2023.100345.

Richards, Timothy J., Mark R. Manfredo, and Dwight R. Sanders. 2004. "Pricing Weather Derivatives." *American Journal of Agricultural Economics* 86 (4): 1005–17. https://doi.org/10.1111/j.0002-9092.2004.00649.x.

Skees, Jerry R. 2008. "Innovations in Index Insurance for the Poor in Lower Income Countries." *Agricultural and Resource Economics Review* 37 (1): 1–15. https://doi.org/10.1017/s1068280500002094.

Taib, Che Mohd Imran Che, and Fred Espen Benth. 2012. "Pricing of Temperature Index Insurance." *Review of Development Finance* 2 (1): 22–31. https://doi.org/10.1016/j.rdf.2012.01.004.

World Bank. 2011. "Weather Index Insurance for Agriculture: Guidance for Development Practitioners." 50. Agriculture and Rural Development Discussion Paper. Washington, DC: The World Bank. https://doi.org/10.1596/26889.

———. 2023. "Index-Based Weather Derivative: Product Note." Washington, DC: World Bank. https://thedocs.worldbank.org/en/doc/29d15a97a9791b5e6cfac1c302d08f08-0340012023/original/product-note-index-based-weather-derivative.pdf.

Xu, Wei, Martin Odening, and Oliver Musshoff. 2008. "Indifference Pricing of Weather Derivatives." *American Journal of Agricultural Economics* 90 (4): 979–93. https://doi.org/10.1111/j.1467-8276.2008.01154.x.

A Downloads

Downloads

You can download the full book or individual chapters as standalone PDF documents below.

A.1 Full Book

Download Full Book (PDF)

A.2 Individual Chapters

If you only need a specific section, select a chapter below:

- Capstone
- GitHub Setup
- Project Proposal
- Python Testing
- Interactive Python Notebook Part 1

> **Note**
>
> These standalone PDFs are automatically updated every time the team pushes new code to GitHub.

B GitHub Setup

Steps to setting up this GitHub Repository

B.1 Created a new repository on GitHub

a. Logged into GitHub and clicked on the "Create new..." icon
b. Selected "New repository" and specified owner, name, and description
c. Chose public visibility, turned on README, and added the R .gitignore template
d. Clicked on Create new repository, then copied the repository URL

B.2 Cloned to a local directory and set up repository as an R project

a. Cloned repository with RStudio to create an associated .Rproj file by navigating to File > New Project > Version Control > Git and then pasting the repository URL
b. Staged, committed, and pushed the new .Rproj file

B.3 Prepared use of Quarto in the repository

a. Added a new line for /.quarto/ to the .gitignore to ignore temporary files generated during the rendering process and remove the line that ignores docs/
b. Created a simple _quarto.yml and index.qmd files
c. Staged, committed, and pushed the 2 new files and the changes to the .gitignore file

B.4 Rendered html and pdf

a. Restarted RStudio for the Build tab to appear
b. In the build tab, selected Render Book > All Formats
c. Staged, committed, and pushed the new docs folder

B.5 Configured GitHub pages

a. Navigated to Pages in the Repository settings on GitHub
b. Selected and saved the main Branch and /docs folder as the source
c. Refreshed the browser and copied the URL

B.6 Created a new chapter in the book

a. Created a new quarto file called github.qmd in the R Studio project
b. Added the name of the new GitHub quarto file to the _quarto.yml list of chapters
c. In the build tab, selected Render Book > All Formats
d. Staged, committed, and pushed the new github.qmd file along with the updated /docs and _quarto.yml file

B.7 Protected the main branch and required all future changes to be submitted via pull requests

a. Configured required use of pull requests to prevent future merge conflicts by navigating to Settings > Branches > Add classic branch protection rule and then specifying the branch name pattern "main" along with selecting "Require a pull request before merging"

B.8 Prepare the repository to run python when engine is specified as jupyter

a. Download python
b. Set a Default Python Interpreter in RStudio
c. Install Jupyter Kernel and the PyYAML via terminal: `pip install PyYAML`
d. Install Jupyter Kernel via termina: `pip install jupyter`
e. Install libraries via termina: `pip install numpy pandas matplotlib seaborn`
f. Create a test_python.qmd file, include: `engine: jupyter`

B.9 Set up a github action to automatically render the project's html and pdf files

C Test Python

This chapter demonstrates running Python code using the Jupyter engine.

C.1 Python Code Chunk

The following code chunk calculates and prints a result.

```
# Import a library and perform a calculation
import numpy as np

# Create an array and calculate the mean
data = np.array([1, 2, 3, 4, 5])
data_mean = np.mean(data)

# Print the result
print(f"The mean of the data is: {data_mean}")
```

```
The mean of the data is: 3.0
```

D Python Notebook Part 1

Part 1: Fishing and Climate Data Analysis & Construction of Fishable Days Index (FDI)

D.1 Import Packages & Set Constants

```
# Import packages
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import seaborn as sns # statistical visualization library

# Sparse daily fishing reports file path
FISHING_PATH = "../data/stx_com_csl_daily_8623.csv"

# Hourly ERA5 wind components
WIND_PATH = "../data/st_croix_wind_data.csv"

# Hourly ERA5 significant wave height
WAVE_PATH = "../data/st_croix_wave_data.csv"

# Annual VI population
POPULATION_PATH = "../data/usvi_pop.csv"

# Date range
START_DATE = "1985-01-01" # first day of study period
END_DATE   = "2024-12-28" # last day of study period

# Unit conversion
MS_TO_KNOTS = 1.94384449 # m/s to knots conversion factor

# Fishable day thresholds (initial values, tunable in Section 9)
```

```
WIND_THRESHOLD_MS = 8.5 #10.0 # max wind speed in m/s (~19.44 knots)
WAVE_THRESHOLD_M  = 1.75 #2.0 # max significant wave height in meters

# Population scaling denominator
POP_SCALE = 1000 # rates per 1k people
```

D.2 Load Raw Climate and Fishing Data

```
# Fishing data: read daily fishing reports, parse date column to datetime
df_fishing = pd.read_csv(
  FISHING_PATH,
  parse_dates=["date"],
  date_format="%m/%d/%Y"
  )

# Keep only relevant columns
df_fishing = df_fishing[["date", "weekday", "n_trips", "n_csl_trips",
                         "all_all", "csl_all"]]

# Wind data read hourly wind, parses timestamp column
df_wind = pd.read_csv(WIND_PATH, parse_dates=["valid_time"])

# Keep only relevant columns
df_wind = df_wind[["valid_time", "u10", "v10", "fg10"]]

# Wave data: read hourly wave heights, parses timestamp column
df_wave = pd.read_csv(WAVE_PATH, parse_dates=["valid_time"])

# Keep only relevant columns
df_wave = df_wave[["valid_time", "swh"]]

# Population data: read annual population, parse date column
df_pop = pd.read_csv(POPULATION_PATH, parse_dates=["observation_date"])

# Rename columns
df_pop = df_pop.rename(columns={"observation_date": "year",
                                "POPTOTVIA647NWDB": "population"})

# Extract integer year from date for merging later
```

```
df_pop["year"] = df_pop["year"].dt.year

# Check
print("Fishing rows:", len(df_fishing)) # should be less than 14,608
print("Wind rows:",    len(df_wind))    # should be ~350,000+ hourly rows
print("Wave rows:",    len(df_wave))    # should be ~350,000+ hourly rows
print("Population rows:", len(df_pop)) # should be 40 rows, one per year
```

```
Fishing rows: 12834
Wind rows: 350568
Wave rows: 350568
Population rows: 40
```

D.3 Construct Completed Time-Series to Fill Non-Reported Dates From Fishing Dataset

```
# Build full dataset structure filling fishing days not reported: 1 row per day
all_dates = pd.DataFrame(
    {"date": pd.date_range(start=START_DATE, end=END_DATE, freq="D")}
)

all_dates["date"] = pd.to_datetime(all_dates["date"], format="%m/%d/%Y")
df_fishing["date"] = pd.to_datetime(df_fishing["date"], format="%m/%d/%y")

# Left join fishing data to full dataset structure: unreported  days become NaN
df_calendar = all_dates.merge(df_fishing, on="date", how="left")

# Fill silent day numeric columns with 0
numeric_cols = ["n_trips", "n_csl_trips", "all_all", "csl_all"]
df_calendar[numeric_cols] = df_calendar[numeric_cols].fillna(0)

# Get weekday from date using pandas dt accessor; overrides sparse file value
df_calendar["weekday"] = df_calendar["date"].dt.day_name()

# Add helper integer year column for population join
df_calendar["year"]  = df_calendar["date"].dt.year

# Add helper integer month column for wind & wave join
df_calendar["month"] = df_calendar["date"].dt.month
```

```
# Check
print("Calendar rows:", len(df_calendar))  # should be exactly 14,608
print("Silent days (no trips reported):",
      (df_calendar["n_trips"] == 0).sum()) # days with zero reports

# Preview first 5 rows
print(df_calendar.head(5))
```

```
Calendar rows: 14607
Silent days (no trips reported): 1773
        date    weekday  n_trips  n_csl_trips  all_all  csl_all  year  month
0 1985-01-01    Tuesday     13.0          0.0    438.0      0.0  1985      1
1 1985-01-02  Wednesday     16.0          0.0    625.0      0.0  1985      1
2 1985-01-03   Thursday     16.0          0.0    565.0      0.0  1985      1
3 1985-01-04     Friday     19.0          0.0    894.0      0.0  1985      1
4 1985-01-05   Saturday     27.0          0.0   1014.0      0.0  1985      1
```

D.4 Process Wind Dataset

```
# Calculate scalar wind speed from U and V components in m/s; (u^2 + v^2)^(1/2)
df_wind["wind_speed_ms"] = np.sqrt(df_wind["u10"]**2 + df_wind["v10"]**2)

# Convert scalar wind speed & gust --> knots
df_wind["wind_speed_kt"] = df_wind["wind_speed_ms"] * MS_TO_KNOTS

# Convert scalar wind speed & gust --> knots
df_wind["wind_gust_kt"]  = df_wind["fg10"]  * MS_TO_KNOTS

# Extract date from hourly timestamp for grouping: strip time component
df_wind["date"] = df_wind["valid_time"].dt.normalize()

# Aggregate hourly to daily maximum
df_wind_daily = (
    df_wind.groupby("date")
    .agg(
        # daily max scalar wind speed in m/s
        max_wind_ms = ("wind_speed_ms", "max"),
        # daily max scalar wind speed in knots
        max_wind_kt = ("wind_speed_kt", "max"),
```

```
        # daily max gust in m/s
        max_gust_ms = ("fg10",          "max"),
        # daily max gust in knots
        max_gust_kt = ("wind_gust_kt",  "max"),
    )
    # bring date back as a regular column
    .reset_index()
)

# Check
print("Daily wind rows:", len(df_wind_daily)) # should be ~14,608
print("Max wind speed recorded (m/s):", df_wind_daily["max_wind_ms"].max(),3)
print("Max wind speed recorded (kt):",  df_wind_daily["max_wind_kt"].max(),3)

# Preview first 5 rows
print(df_wind_daily.head(5))
```

```
Daily wind rows: 14607
Max wind speed recorded (m/s): 28.639624763784898 3
Max wind speed recorded (kt): 55.67097679275083 3
        date  max_wind_ms  max_wind_kt  max_gust_ms  max_gust_kt
0 1985-01-01    10.975895    21.335433    15.157806    29.464418
1 1985-01-02    10.659820    20.721032    14.236787    27.674100
2 1985-01-03    10.643230    20.688784    14.280316    27.758714
3 1985-01-04     9.447021    18.363539    12.966755    25.205355
4 1985-01-05     6.931456    13.473673     9.752116    18.956597
```

D.5 Process Wave Dataset

```
# Extract date from hourly timestamp column for grouping: strip time component
df_wave["date"] = df_wave["valid_time"].dt.normalize()

# Aggregate the hourly to daily maximums
df_wave_daily = (
    df_wave.groupby("date")
    .agg(
        # daily max. swh in metres
        max_swh_m = ("swh", "max"),
    )
```

```
    # brings date back as regular column
    .reset_index()
)

# Check
print("Daily wave rows:", len(df_wave_daily)) # should be ~14,608
print("Max wave height recorded (m):",
      round(df_wave_daily["max_swh_m"].max(), 3))
print("Min wave height recorded (m):",
      round(df_wave_daily["max_swh_m"].min(), 3))

# Preview first 3 rows
print(df_wave_daily.head(3))
```

```
Daily wave rows: 14607
Max wave height recorded (m): 9.329
Min wave height recorded (m): 0.539
        date  max_swh_m
0 1985-01-01   2.061214
1 1985-01-02   1.997108
2 1985-01-03   2.103769
```

D.6 Merge Wind/Wave/Population Datasets into Fishing Dataset

```
# Join wind into completed dates frame: left join kept all >14k calendar days
df_master = df_calendar.merge(df_wind_daily, on="date", how="left")

# Join wave into completed dates frame: retain all calendar days
df_master = df_master.merge(df_wave_daily, on="date", how="left")

# Join pop. onto completed dates by  integer year; days assigned annual pop.
df_master = df_master.merge(df_pop, on="year", how="left")

# Check for missing climate data after the merges: days with no wind data
wind_missing = df_master["max_wind_ms"].isna().sum()

# Check for missing climate data after the merges: days with no wave data
wave_missing = df_master["max_swh_m"].isna().sum()
```

```
# Check for missing climate data after the merges: days with no population data
pop_missing  = df_master["population"].isna().sum()

print("Days missing wind data:", wind_missing)
print("Days missing wave data:", wave_missing)
print("Days missing population data:", pop_missing)

# Fill missing climate data as a conservative gap: fwd last known wind value
df_master["max_wind_ms"] = df_master["max_wind_ms"].ffill()
df_master["max_wind_kt"] = df_master["max_wind_kt"].ffill()
df_master["max_gust_ms"] = df_master["max_gust_ms"].ffill()
df_master["max_gust_kt"] = df_master["max_gust_kt"].ffill()

# Fill missing climate data as a conservative gap: fwd last known wave value
df_master["max_swh_m"]   = df_master["max_swh_m"].ffill()

# Reorder: Final completed column order
df_master = df_master[["date", "year", "month", "weekday",
                       "n_trips", "n_csl_trips", "all_all", "csl_all",
                       "max_wind_ms", "max_wind_kt",
                       "max_gust_ms", "max_gust_kt",
                       "max_swh_m", "population"]]

# Save as csv
import os
from pathlib import Path
data_folder = Path(os.getcwd()).parent / "data"
df_master.to_csv(data_folder / "df_master.csv", index=False)

# Check
print("\nMaster table shape:", df_master.shape) # should be (14608, 14)

# Preview first 3 rows
print(df_master.head(3))
```

```
Days missing wind data: 0
Days missing wave data: 0
Days missing population data: 0

Master table shape: (14607, 14)
        date  year  month    weekday  n_trips  n_csl_trips  all_all  csl_all  \
0 1985-01-01  1985      1    Tuesday     13.0          0.0    438.0      0.0
```

```
1 1985-01-02  1985      1  Wednesday     16.0          0.0    625.0      0.0
2 1985-01-03  1985      1   Thursday     16.0          0.0    565.0      0.0

   max_wind_ms  max_wind_kt  max_gust_ms  max_gust_kt  max_swh_m  population
0    10.975895    21.335433    15.157806    29.464418   2.061214      100760
1    10.659820    20.721032    14.236787    27.674100   1.997108      100760
2    10.643230    20.688784    14.280316    27.758714   2.103769      100760
```

D.7 Population Adjustment (per 1k people)

```
# Lobster trips per 1k people: normalized lobster trip rate
df_master["csl_trips_per_1k"] = (
  (df_master["n_csl_trips"] / df_master["population"]) * POP_SCALE
  )

# Total trips per 1kpeople: normalised total trip rate
df_master["all_trips_per_1k"] = (
  (df_master["n_trips"] / df_master["population"]) * POP_SCALE
  )

# Lobster lbs caught per 1k people: normalised lobster catch rate
df_master["csl_lbs_per_1k"] = (
  (df_master["csl_all"] / df_master["population"]) * POP_SCALE
  )

# Total lbs caught per 1k people: normalised total catch rate
df_master["all_lbs_per_1k"] = (
  (df_master["all_all"] / df_master["population"]) * POP_SCALE
  )

# Check
print("Max csl trips per 1k:", round(df_master["csl_trips_per_1k"].max(), 4))
print("Mean csl trips per 1k on days with any trips:",
      round(df_master.loc[df_master["n_csl_trips"] > 0,
      "csl_trips_per_1k"].mean(), 4)) # average only over active fishing days

# Include silent days and genuinely zero lobster days
print("Days with zero csl trips:", (df_master["n_csl_trips"] == 0).sum())

# Preview 5 rows of most important columns
```

```
print(df_master[["date", "n_csl_trips", "csl_trips_per_1k",
                 "csl_all", "csl_lbs_per_1k"]].head(5))
```

```
Max csl trips per 1k: 0.2215
Mean csl trips per 1k on days with any trips: 0.0678
Days with zero csl trips: 6423
        date  n_csl_trips  csl_trips_per_1k  csl_all  csl_lbs_per_1k
0 1985-01-01          0.0               0.0      0.0             0.0
1 1985-01-02          0.0               0.0      0.0             0.0
2 1985-01-03          0.0               0.0      0.0             0.0
3 1985-01-04          0.0               0.0      0.0             0.0
4 1985-01-05          0.0               0.0      0.0             0.0
```

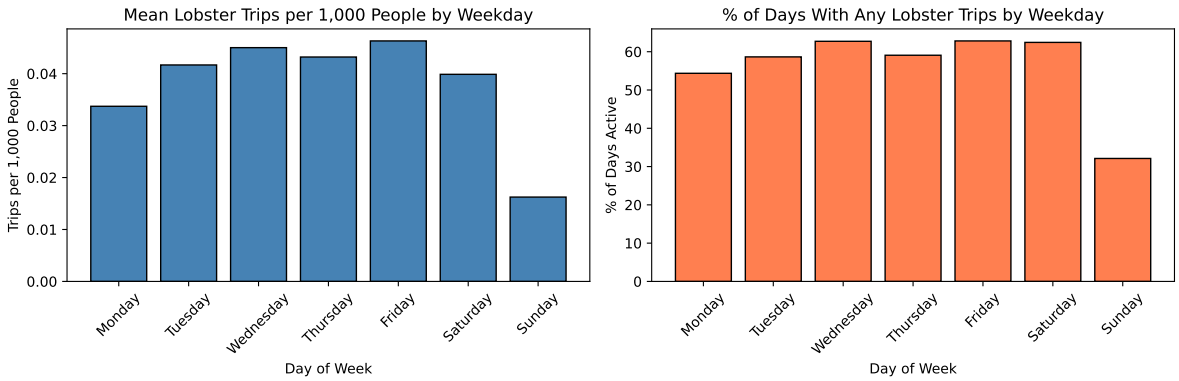
D.8 Check for Weekday Bias in Fishing Trips

```
# Define an ordered weekday list for consistent plotting
weekday_order = ["Monday", "Tuesday", "Wednesday", "Thursday",
                 "Friday", "Saturday", "Sunday"]

# Calculats mean population-adjusted lobster metrics by weekday
weekday_bias = (
    df_master.groupby("weekday")
    .agg(
        # avg normalised lobster trips by weekday
        mean_csl_trips_per_1k = ("csl_trips_per_1k", "mean"),
        # avg normalised lobster lbs by weekday
        mean_csl_lbs_per_1k   = ("csl_lbs_per_1k",   "mean"),
        # % of days that had any lobster trips
        pct_days_with_trips   = ("n_csl_trips", lambda x: (x > 0).mean() * 100)
    )
    # enforce Monday-Sunday order
    .reindex(weekday_order)
    .reset_index()
)

# print full table without row index
print(weekday_bias.to_string(index=False))
```

```
  weekday  mean_csl_trips_per_1k  mean_csl_lbs_per_1k  pct_days_with_trips
```

Monday	0.033722	1.238984	54.362416
Tuesday	0.041677	1.493190	58.648778
Wednesday	0.045011	1.642619	62.721610
Thursday	0.043207	1.572531	59.080019
Friday	0.046304	1.747825	62.817441
Saturday	0.039874	1.337318	62.434116
Sunday	0.016240	0.549767	32.118888

```
# Subplot 1: mean lobster trips per 1k ppl by weekday
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# bar chart of mean trips
axes[0].bar(weekday_bias["weekday"], weekday_bias["mean_csl_trips_per_1k"],
            color="steelblue", edgecolor="black")
axes[0].set_title("Mean Lobster Trips per 1,000 People by Weekday")
axes[0].set_xlabel("Day of Week")
axes[0].set_ylabel("Trips per 1,000 People")
# rotate x labels for readability
axes[0].tick_params(axis="x", rotation=45)

# Subplot 2: % of days with any lobster trips: activity freq. by weekday
axes[1].bar(weekday_bias["weekday"], weekday_bias["pct_days_with_trips"],
            color="coral", edgecolor="black")
axes[1].set_title("% of Days With Any Lobster Trips by Weekday")
axes[1].set_xlabel("Day of Week")
axes[1].set_ylabel("% of Days Active")
axes[1].tick_params(axis="x", rotation=45)

plt.tight_layout()
plt.show()
```



```
# Flag Sundays for exclusion from climate threshold validation
df_master["is_sunday"] = (df_master["weekday"] == "Sunday").astype(int)
print("Sunday days flagged:", df_master["is_sunday"].sum()) # should be ~2,087
```

```
Sunday days flagged: 2086
```

D.9 Defining the Fishable Day Index (FDI)

```
# Apply climate thresholds defining fishable days: 1 if wind below threshold
df_master["wind_ok"] = (
  (df_master["max_wind_ms"] <= WIND_THRESHOLD_MS).astype(int)
)

# Apply thresholds defining fishable days: 1 if wave height below threshold
df_master["wave_ok"] = (
  (df_master["max_swh_m"]   <= WAVE_THRESHOLD_M).astype(int)
  )

# 1 if BOTH conditions met; wind & wave observations define fishablye/unfishable
df_master["fdi"] = (df_master["wind_ok"] & df_master["wave_ok"]).astype(int)

# Printout of summary of FDI classification: counts of FDI = 1 days
print("Total fishable days:",   df_master["fdi"].sum())

# Printout of summary of FDI classification: counts of FDI = 0 days
print("Total unfishable days:", (df_master["fdi"] == 0).sum())

# Printout of summary of FDI classification: overall count for the fishable rate
print("% fishable:", round(df_master["fdi"].mean() * 100, 2))

# Validation; compare activity fishable vs unfishable days; drop Sundays
df_validation = df_master[df_master["is_sunday"] == 0].copy()

validation_summary = (
    df_validation.groupby("fdi")
    .agg(
        # mean norm. lobster trips
        mean_csl_trips_per_1k = ("csl_trips_per_1k", "mean"),
        # mean norm. lobster lbs
```

```
        mean_csl_lbs_per_1k   = ("csl_lbs_per_1k",   "mean"),
        # % of days with any lobster activity
        pct_days_active       = ("n_csl_trips", lambda x: (x > 0).mean() * 100)
    )
    .reset_index()
)
# Label for readability
validation_summary["fdi"] = (
  validation_summary["fdi"].map({0: "Unfishable", 1: "Fishable"})
  )
print("\nValidation Summary (Sundays excluded):")
print(validation_summary.to_string(index=False))

# Analysis for the threshold sensitivity: range of wind thresholds in m/s
wind_thresholds = [7.5, 8.5, 10.0, 11.5, 13.0]

# Analysis for the threshold sensitivity: range of wave thresholds in meters
wave_thresholds = [1.5, 1.75, 2.0, 2.25, 2.5]

sensitivity_rows = []
for wnd in wind_thresholds:
    for wav in wave_thresholds:
        # computes FDI for each threshold combination
        fdi_temp = ((df_master["max_wind_ms"] <= wnd) &
                    (df_master["max_swh_m"]   <= wav)).astype(int)
        sensitivity_rows.append({
            "wind_thresh_ms": wnd,
            # converted to knots for reference
            "wind_thresh_kt": round(wnd * MS_TO_KNOTS, 3),
            "wave_thresh_m":  wav,
            "fishable_days":  fdi_temp.sum(),
            "pct_fishable":   round(fdi_temp.mean() * 100, 2)
        })

# Collect all combinations into a new panddas dataframe
df_sensitivity = pd.DataFrame(sensitivity_rows)
print("\nThreshold Sensitivity Analysis:")
print(df_sensitivity.to_string(index=False))
```

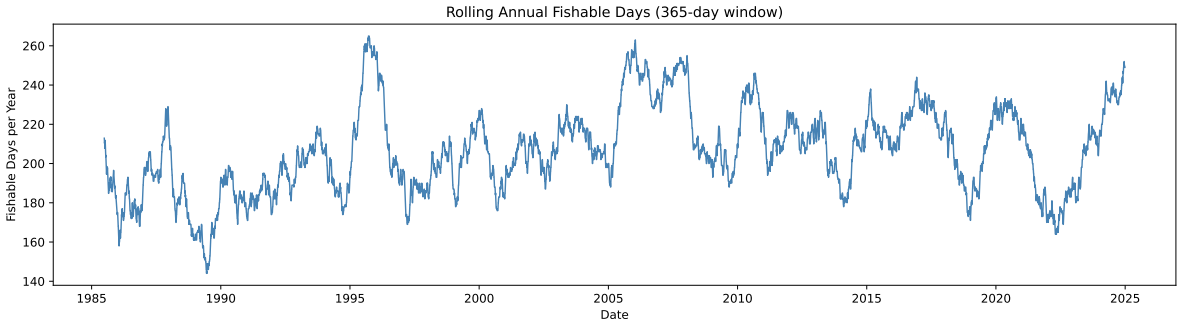
```
Total fishable days: 8278
Total unfishable days: 6329
% fishable: 56.67
```

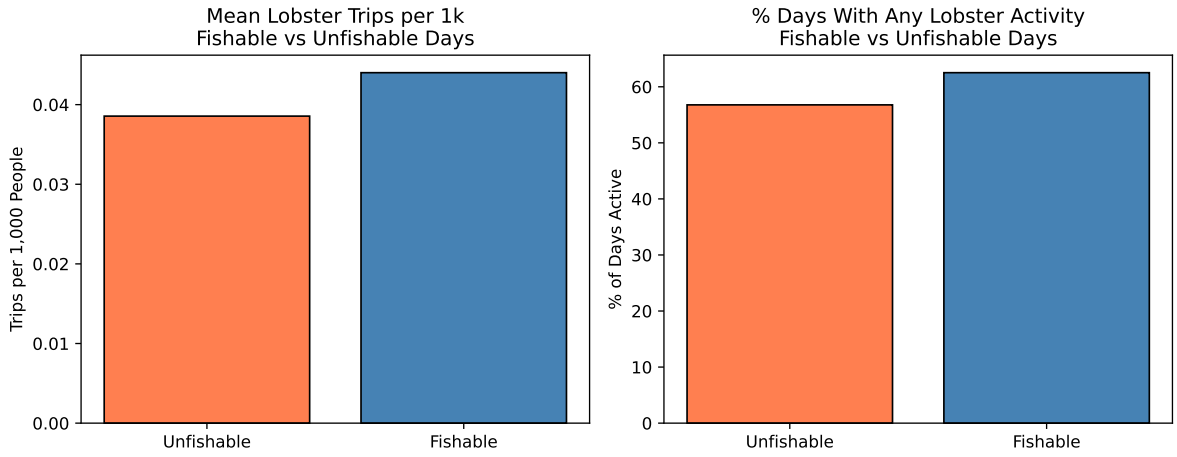
```
Validation Summary (Sundays excluded):
       fdi  mean_csl_trips_per_1k  mean_csl_lbs_per_1k  pct_days_active
Unfishable                0.038558             1.400545        56.774075
  Fishable                0.044012             1.586591        62.515937

Threshold Sensitivity Analysis:
 wind_thresh_ms  wind_thresh_kt  wave_thresh_m  fishable_days  pct_fishable
            7.5          14.579           1.50           4872         33.35
            7.5          14.579           1.75           5048         34.56
            7.5          14.579           2.00           5087         34.83
            7.5          14.579           2.25           5094         34.87
            7.5          14.579           2.50           5099         34.91
            8.5          16.523           1.50           7697         52.69
            8.5          16.523           1.75           8278         56.67
            8.5          16.523           2.00           8372         57.31
            8.5          16.523           2.25           8389         57.43
            8.5          16.523           2.50           8395         57.47
           10.0          19.438           1.50           9143         62.59
           10.0          19.438           1.75          11707         80.15
           10.0          19.438           2.00          12505         85.61
           10.0          19.438           2.25          12620         86.40
           10.0          19.438           2.50          12636         86.51
           11.5          22.354           1.50           9151         62.65
           11.5          22.354           1.75          11929         81.67
           11.5          22.354           2.00          13377         91.58
           11.5          22.354           2.25          14053         96.21
           11.5          22.354           2.50          14229         97.41
           13.0          25.270           1.50           9151         62.65
           13.0          25.270           1.75          11930         81.67
           13.0          25.270           2.00          13390         91.67
           13.0          25.270           2.25          14101         96.54
           13.0          25.270           2.50          14384         98.47
```

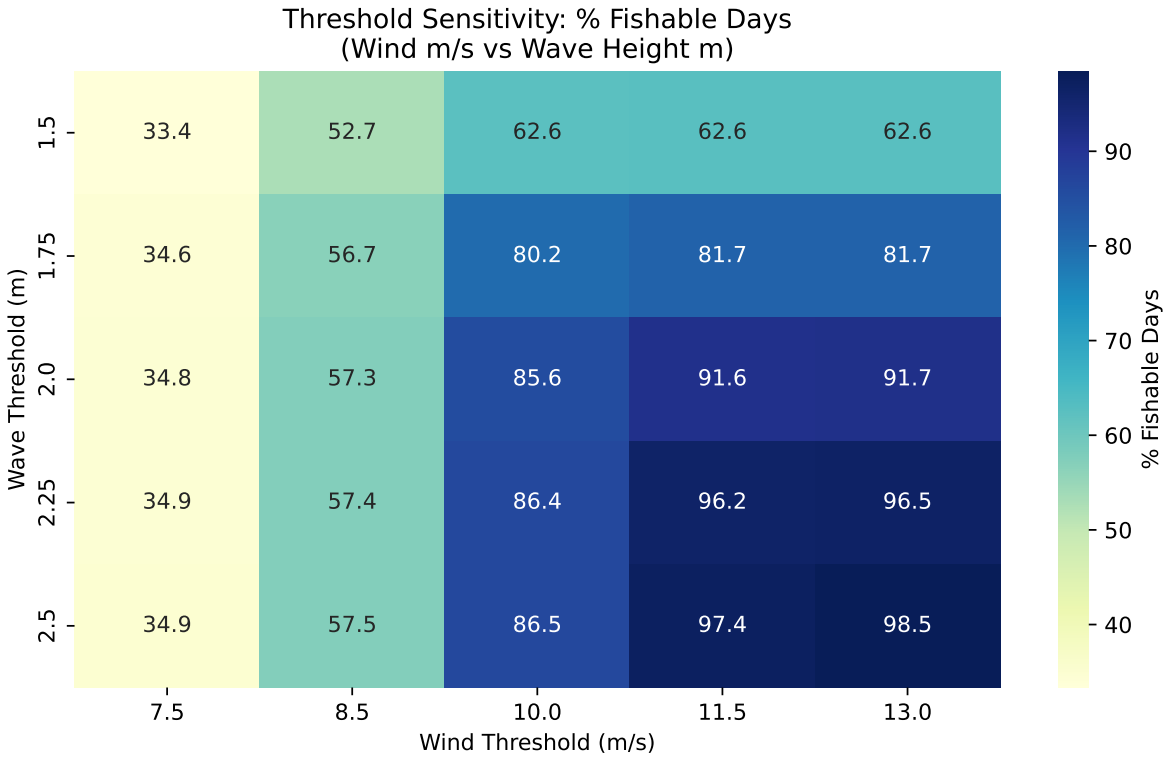
```python
# Subplot 1: FDI daily time series (annual rolling average for readability)
df_master["fdi_rolling"] = (
  df_master["fdi"].rolling(window=365, min_periods=180).mean() * 365
  ) # rolling annual fishable days
fig, ax = plt.subplots(figsize=(14, 4))
ax.plot(df_master["date"], df_master["fdi_rolling"],
        color="steelblue", linewidth=1.2)
ax.set_title("Rolling Annual Fishable Days (365-day window)")
```

```
ax.set_xlabel("Date")
ax.set_ylabel("Fishable Days per Year")
# Clean integer y axis labels
ax.yaxis.set_major_formatter(ticker.FormatStrFormatter("%.0f"))
plt.tight_layout()
plt.show()

# Subplot 2: validation bar chart
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

axes[0].bar(validation_summary["fdi"],
            validation_summary["mean_csl_trips_per_1k"],
            color=["coral", "steelblue"], edgecolor="black")
axes[0].set_title("Mean Lobster Trips per 1k\nFishable vs Unfishable Days")
axes[0].set_ylabel("Trips per 1,000 People")
axes[0].set_xlabel("")

axes[1].bar(validation_summary["fdi"], validation_summary["pct_days_active"],
            color=["coral", "steelblue"], edgecolor="black")
axes[1].set_title(
  "% Days With Any Lobster Activity\nFishable vs Unfishable Days"
  )
axes[1].set_ylabel("% of Days Active")
axes[1].set_xlabel("")

plt.tight_layout()
plt.show()

# Subplot3: sensitivity heatmap: reshape for heatmap format
df_heatmap = df_sensitivity.pivot(index="wave_thresh_m",
                                  columns="wind_thresh_ms",
                                  values="pct_fishable")
fig, ax = plt.subplots(figsize=(8, 5))

# Annotated heatmap of threshold combinations
sns.heatmap(df_heatmap, annot=True, fmt=".1f", cmap="YlGnBu",
            ax=ax, cbar_kws={"label": "% Fishable Days"})
ax.set_title(
  "Threshold Sensitivity: % Fishable Days\n(Wind m/s vs Wave Height m)"
)
ax.set_xlabel("Wind Threshold (m/s)")
ax.set_ylabel("Wave Threshold (m)")
```

```
plt.tight_layout()
plt.show()
```







D.10 Summary of Seasonal FDI

```
# Aggregated daily FDI to the annual fishable day count
df_seasonal = (
    df_master.groupby("year")
    .agg(
        # total days in each year
        total_days        = ("fdi", "count"),
        # count of FDI = 1 days
        fishable_days     = ("fdi", "sum"),
        # count of FDI = 0 days
        unfishable_days   = ("fdi", lambda x: (x == 0).sum()),
        # % of year that was fishable
        pct_fishable      = ("fdi", lambda x: round(x.mean() * 100, 2)),
        # annual mean of daily max wind
        mean_wind_ms      = ("max_wind_ms", "mean"),
        # annual mean of daily max wave height
```

```
        mean_wave_m       = ("max_swh_m",   "mean"),
        # days with any lobster trips reported
        active_fishing_days = ("n_csl_trips", lambda x: (x > 0).sum())
    )
    .reset_index()
)
```

```
# Added lobster activity rate on fishable vs all days
fishable_active = (
    df_master[df_master["fdi"] == 1]
    .groupby("year")
     # lobster active days that were also fishable
    .agg(fishable_active_days = ("n_csl_trips", lambda x: (x > 0).sum()))
    .reset_index()
)
# Join back onto seasonal summary
df_seasonal = df_seasonal.merge(fishable_active, on="year", how="left")

# Mean annual fishable days across full period
long_run_mean = df_seasonal["fishable_days"].mean()

# Standard deviation of annual fishable days
long_run_std  = df_seasonal["fishable_days"].std()

# Calculate the long run mean and strike level reference
strike_90     = long_run_mean * 0.90 # 90% of mean as indicative strike level
strike_80     = long_run_mean * 0.80 # 80% of mean as indicative strike level
```

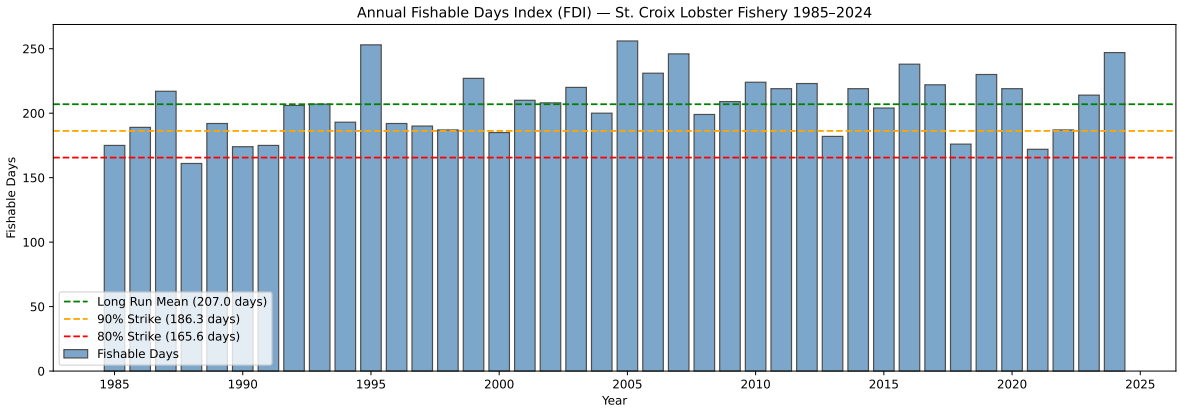
```
# 1. Calculate the values first
years_below_90 = (df_seasonal['fishable_days'] < strike_90).sum()
years_below_80 = (df_seasonal['fishable_days'] < strike_80).sum()

# 2. Print them using simple f-strings
print(f"Long run mean fishable days:      {round(long_run_mean, 1)}")
print(f"Long run std dev:                 {round(long_run_std, 1)}")
print(f"Indicative strike at 90% of mean: {round(strike_90, 1)} days")
print(f"Indicative strike at 80% of mean: {round(strike_80, 1)} days")

print(f"\nYears below 90% strike: {years_below_90}")
print(f"Years below 80% strike: {years_below_80}")

print(f"\nSeasonal FDI Summary:")
```

```
print(df_seasonal[["year", "fishable_days", "pct_fishable",
                   "mean_wind_ms", "mean_wave_m"]].to_string(index=False))
```

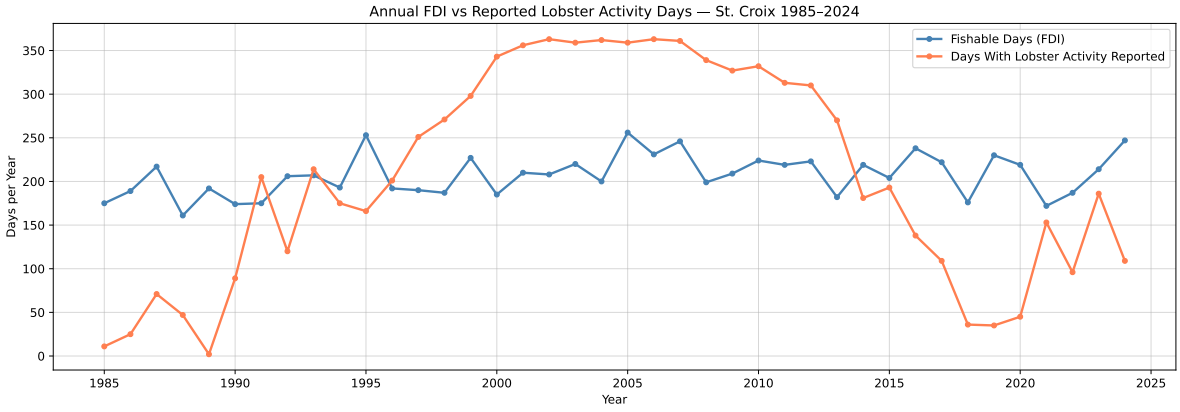
```
Long run mean fishable days:      207.0
Long run std dev:                 23.9
Indicative strike at 90% of mean: 186.3 days
Indicative strike at 80% of mean: 165.6 days

Years below 90% strike: 8
Years below 80% strike: 1

Seasonal FDI Summary:
 year  fishable_days  pct_fishable  mean_wind_ms  mean_wave_m
 1985            175         47.95      8.378711     1.402619
 1986            189         51.78      8.209939     1.384351
 1987            217         59.45      7.966806     1.337351
 1988            161         43.99      8.540703     1.487091
 1989            192         52.60      8.358651     1.426431
 1990            174         47.67      8.393137     1.438188
 1991            175         47.95      8.457029     1.460763
 1992            206         56.28      7.966727     1.447969
 1993            207         56.71      8.077101     1.442734
 1994            193         52.88      8.212212     1.450661
 1995            253         69.32      7.695674     1.405975
 1996            192         52.46      8.383011     1.537843
 1997            190         52.05      8.382274     1.458454
 1998            187         51.23      8.385344     1.512378
 1999            227         62.19      8.088638     1.471468
 2000            185         50.55      8.306767     1.489493
 2001            210         57.53      8.071573     1.427403
 2002            208         56.99      8.196841     1.437431
 2003            220         60.27      7.847460     1.404625
 2004            200         54.64      8.166434     1.444430
 2005            256         70.14      7.572696     1.339890
 2006            231         63.29      7.885105     1.406349
 2007            246         67.40      7.798133     1.358193
 2008            199         54.37      8.245858     1.478545
 2009            209         57.26      8.053282     1.449005
 2010            224         61.37      7.788298     1.414063
 2011            219         60.00      7.868040     1.413033
 2012            223         60.93      7.945315     1.402381
 2013            182         49.86      8.323493     1.485063
```

2014	219	60.00	8.017828	1.416118
2015	204	55.89	8.091917	1.441943
2016	238	65.03	7.798298	1.420609
2017	222	60.82	8.099177	1.494537
2018	176	48.22	8.393065	1.566984
2019	230	63.01	7.750875	1.382264
2020	219	59.84	7.931521	1.425647
2021	172	47.12	8.388025	1.455910
2022	187	51.23	8.345445	1.485085
2023	214	58.63	7.832936	1.410504
2024	247	68.04	7.617995	1.333671

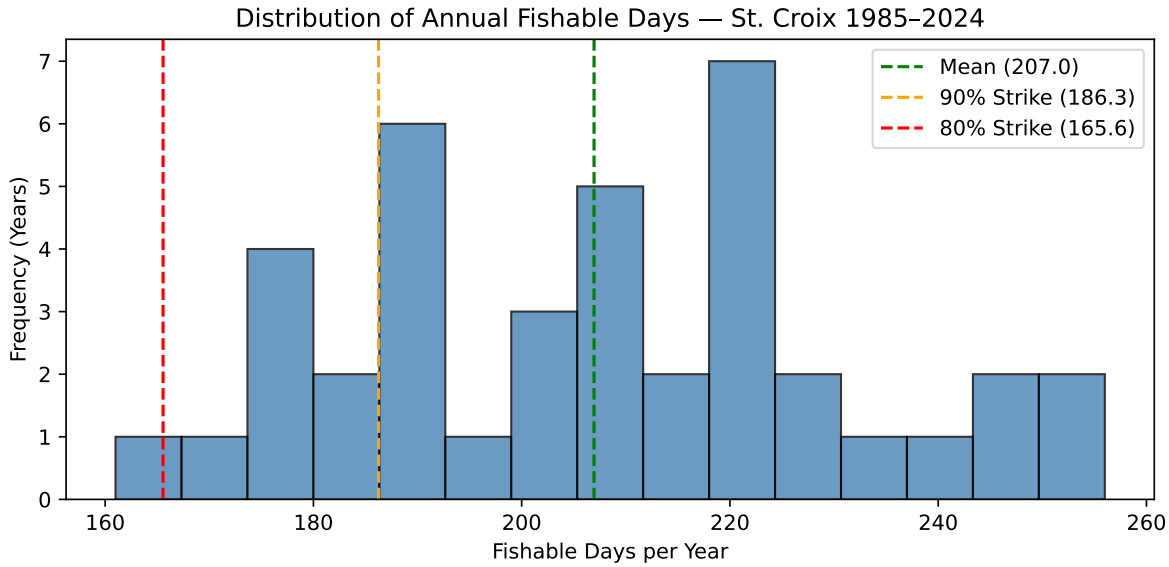
```
# Sublot 1: annual fishable days time series with 80/90 mean strike levels
fig, ax = plt.subplots(figsize=(14, 5))

# annual bar chart
ax.bar(df_seasonal["year"], df_seasonal["fishable_days"],
       color="steelblue", edgecolor="black", alpha=0.7, label="Fishable Days")

# mean ref. line
ax.axhline(long_run_mean, color="green",  linewidth=1.5, linestyle="--",
           label=f"Long Run Mean ({round(long_run_mean, 1)} days)")

# 90% strike ref. line
ax.axhline(strike_90,     color="orange", linewidth=1.5, linestyle="--",
           label=f"90% Strike ({round(strike_90, 1)} days)")
# 80% strike ref. line
ax.axhline(strike_80,     color="red",    linewidth=1.5, linestyle="--",
           label=f"80% Strike ({round(strike_80, 1)} days)")

ax.set_title(
  "Annual Fishable Days Index (FDI) - St. Croix Lobster Fishery 1985-2024"
  )
ax.set_xlabel("Year")
ax.set_ylabel("Fishable Days")
ax.legend(loc="lower left")
ax.yaxis.set_major_formatter(ticker.FormatStrFormatter("%.0f"))
plt.tight_layout()
plt.show()
```




```
# Subplot 2: FDI vs active lobster fishing days per year
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(df_seasonal["year"], df_seasonal["fishable_days"],
        color="steelblue", linewidth=2, marker="o", markersize=4,
        label="Fishable Days (FDI)")
ax.plot(df_seasonal["year"], df_seasonal["active_fishing_days"],
        color="coral", linewidth=2, marker="o", markersize=4,
        label="Days With Lobster Activity Reported")
ax.set_title(
  "Annual FDI vs Reported Lobster Activity Days - St. Croix 1985-2024"
  )
ax.set_xlabel("Year")
ax.set_ylabel("Days per Year")
ax.legend()
plt.tight_layout()
plt.grid(True,alpha=0.5)
plt.show()
```



```
# Subplot 3: distribution of annual fishable days
fig, ax = plt.subplots(figsize=(8, 4))

# histogram of annual FDI values
ax.hist(df_seasonal["fishable_days"], bins=15, color="steelblue",
        edgecolor="black", alpha=0.8)
ax.axvline(long_run_mean, color="green",  linestyle="--", linewidth=1.5,
           label=f"Mean ({round(long_run_mean, 1)})")
ax.axvline(strike_90,     color="orange", linestyle="--", linewidth=1.5,
           label=f"90% Strike ({round(strike_90, 1)})")
ax.axvline(strike_80,     color="red",    linestyle="--", linewidth=1.5,
           label=f"80% Strike ({round(strike_80, 1)})")
ax.set_title("Distribution of Annual Fishable Days - St. Croix 1985-2024")
ax.set_xlabel("Fishable Days per Year")
ax.set_ylabel("Frequency (Years)")
ax.legend()
plt.tight_layout()
plt.show()
```



D.11 Exporting the Outputs

```
# Export the full daily master table to 2 .csv files
master_path   = "../data/fdi_master_daily.csv"
seasonal_path = "../data/fdi_seasonal_summary.csv"

# save full daily table, no row index
df_master.to_csv(master_path, index=False)

# save annual FDI summary, no row index
df_seasonal.to_csv(seasonal_path, index=False)
```

```
# Checks
print(f"Daily master table exported:    {master_path}")
print(f"  Rows: {len(df_master)}, Columns: {len(df_master.columns)}")
print(f"\nSeasonal FDI summary exported:  {seasonal_path}")
print(f"  Rows: {len(df_seasonal)}, Columns: {len(df_seasonal.columns)}")

# Printout of final columns header of both .csv files: list out every column
print(f"\nDaily master columns:")
for col in df_master.columns:
    print(f"  {col}")

print(f"\nSeasonal summary columns:")
for col in df_seasonal.columns:
    print(f"  {col}")
```

```
Daily master table exported:    ../data/fdi_master_daily.csv
  Rows: 14607, Columns: 23

Seasonal FDI summary exported:  ../data/fdi_seasonal_summary.csv
  Rows: 40, Columns: 9

Daily master columns:
  date
  year
  month
  weekday
  n_trips
  n_csl_trips
  all_all
  csl_all
  max_wind_ms
```

```
  max_wind_kt
  max_gust_ms
  max_gust_kt
  max_swh_m
  population
  csl_trips_per_1k
  all_trips_per_1k
  csl_lbs_per_1k
  all_lbs_per_1k
  is_sunday
  wind_ok
  wave_ok
  fdi
  fdi_rolling

Seasonal summary columns:
  year
  total_days
  fishable_days
  unfishable_days
  pct_fishable
  mean_wind_ms
  mean_wave_m
  active_fishing_days
  fishable_active_days
```

E poisson_binomial

Poisson-binomial using daily historical (Strike in Percent)

E.1 Import Packages and Set File Paths

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

FISHING_PATH    = "../data/stx_com_csl_daily_8623.csv"
WIND_PATH       = "../data/st_croix_wind_data.csv"
WAVE_PATH       = "../data/st_croix_wave_data.csv"
POPULATION_PATH = "../data/usvi_pop.csv"
```

E.2 Historical Data Window and User Inputs

```
START_DATE = "1985-01-01"
END_DATE   = "2024-12-28"

MS_TO_KNOTS = 1.94384449
POP_SCALE = 1000

RISK_FREE_RATE     = 0.037
AVG_PROFIT_PER_DAY = 330.0

# CHOSEN FDI THRESHOLDS
WIND_THRESHOLD_MS = 8.5
WAVE_THRESHOLD_M  = 1.75

CONTRACT_START_MMDD = "01-01"
```

```
CONTRACT_END_MMDD   = "12-31"

# THRESHOLD SENSITIVITY GRID
wind_thresholds = [7.5, 8.5, 10.0, 11.5, 13.0]
wave_thresholds = [1.5, 1.75, 2.0, 2.25, 2.5]

# STRIKE GRID
STRIKE_PERCENTS = np.round(np.arange(0.80, 0.951, 0.01), 2)
```

E.3 LOAD DATA AND SENSITIVITY ANALYSIS

```
# LOAD DATA
df_fishing = pd.read_csv(
  FISHING_PATH,
  parse_dates=["date"],
  date_format="%m/%d/%Y"
  )
df_fishing = df_fishing[["date", "weekday", "n_trips", "n_csl_trips", "all_all", "csl_all"]]

df_wind = pd.read_csv(WIND_PATH, parse_dates=["valid_time"])
df_wind = df_wind[["valid_time", "u10", "v10", "fg10"]].copy()

df_wave = pd.read_csv(WAVE_PATH, parse_dates=["valid_time"])
df_wave = df_wave[["valid_time", "swh"]].copy()

df_pop = pd.read_csv(POPULATION_PATH, parse_dates=["observation_date"])
df_pop = df_pop.rename(columns={
    "observation_date": "year",
    "POPTOTVIA647NWDB": "population"
})
df_pop["year"] = df_pop["year"].dt.year
```

```
# BUILD DAILY MASTER TABLE
dates = pd.date_range(start=START_DATE, end=END_DATE, freq="D")
df_calendar = pd.DataFrame({"date": dates})

df_calendar["year"] = df_calendar["date"].dt.year
df_calendar["weekday"] = df_calendar["date"].dt.day_name()
df_calendar["is_sunday"] = (df_calendar["weekday"] == "Sunday").astype(int)
```

```
df_calendar["date"] = pd.to_datetime(df_calendar["date"], format="%m/%d/%Y")
df_fishing["date"] = pd.to_datetime(df_fishing["date"], format="%m/%d/%y")

df_master = df_calendar.merge(df_fishing, on=["date", "weekday"], how="left")

for col in ["n_trips", "n_csl_trips", "all_all", "csl_all"]:
    df_master[col] = df_master[col].fillna(0)
```

F PROCESS WIND

```
df_wind["wind_speed_ms"] = np.sqrt(df_wind["u10"]**2 + df_wind["v10"]**2)
df_wind["date"] = df_wind["valid_time"].dt.normalize()

df_wind_daily = (
    df_wind.groupby("date")
    .agg(
        max_wind_ms=("wind_speed_ms", "max"),
        mean_wind_ms=("wind_speed_ms", "mean"),
        max_gust_ms=("fg10", "max")
    )
    .reset_index()
)
```

G PROCESS WAVE

```
df_wave["date"] = df_wave["valid_time"].dt.normalize()

df_wave_daily = (
    df_wave.groupby("date")
    .agg(
        max_swh_m=("swh", "max"),
        mean_swh_m=("swh", "mean")
    )
    .reset_index()
)
```

H MERGE ALL

```
df_master = df_master.merge(df_wind_daily, on="date", how="left")
df_master = df_master.merge(df_wave_daily, on="date", how="left")
df_master = df_master.merge(df_pop[["year", "population"]], on="year", how="left")

df_master["csl_trips_per_1k"] = (
  (df_master["n_csl_trips"] / df_master["population"]) * POP_SCALE
  )

df_master["csl_lbs_per_1k"]   = (
  (df_master["csl_all"]     / df_master["population"]) * POP_SCALE
  )
```

```
# THRESHOLD SENSITIVITY

sensitivity_rows = []

for wnd in wind_thresholds:
    for wav in wave_thresholds:
        fdi_temp = ((df_master["max_wind_ms"] <= wnd) &
                    (df_master["max_swh_m"]   <= wav)).astype(int)

        annual_temp = (
            pd.DataFrame({
                "year": df_master["year"],
                "fdi_temp": fdi_temp
            })
            .groupby("year")
            .agg(
                annual_fdi=("fdi_temp", "sum"),
                pct_fishable=("fdi_temp", lambda x: x.mean() * 100)
            )
            .reset_index()
        )
```

```
        sensitivity_rows.append({
            "wind_thresh_ms": wnd,
            "wind_thresh_kt": wnd * MS_TO_KNOTS,
            "wave_thresh_m": wav,
            "pct_fishable_total": fdi_temp.mean() * 100,
            "mean_annual_fdi": annual_temp["annual_fdi"].mean(),
            "std_annual_fdi": annual_temp["annual_fdi"].std()
        })

df_sensitivity = pd.DataFrame(sensitivity_rows)

print("\nThreshold Sensitivity Table:")
print(df_sensitivity.round(2).to_string(index=False))
```

```
Threshold Sensitivity Table:
 wind_thresh_ms  wind_thresh_kt  wave_thresh_m  pct_fishable_total  mean_annual_fdi  std_annu
            7.5           14.58           1.50               33.35           121.80
            7.5           14.58           1.75               34.56           126.20
            7.5           14.58           2.00               34.83           127.18
            7.5           14.58           2.25               34.87           127.35
            7.5           14.58           2.50               34.91           127.48
            8.5           16.52           1.50               52.69           192.42
            8.5           16.52           1.75               56.67           206.95
            8.5           16.52           2.00               57.31           209.30
            8.5           16.52           2.25               57.43           209.72
            8.5           16.52           2.50               57.47           209.88
           10.0           19.44           1.50               62.59           228.58
           10.0           19.44           1.75               80.15           292.68
           10.0           19.44           2.00               85.61           312.62
           10.0           19.44           2.25               86.40           315.50
           10.0           19.44           2.50               86.51           315.90
           11.5           22.35           1.50               62.65           228.78
           11.5           22.35           1.75               81.67           298.23
           11.5           22.35           2.00               91.58           334.42
           11.5           22.35           2.25               96.21           351.32
           11.5           22.35           2.50               97.41           355.72
           13.0           25.27           1.50               62.65           228.78
           13.0           25.27           1.75               81.67           298.25
           13.0           25.27           2.00               91.67           334.75
           13.0           25.27           2.25               96.54           352.52
           13.0           25.27           2.50               98.47           359.60
```

```
# BUILD FINAL FDI

df_master["fdi"] = (
    (df_master["max_wind_ms"] <= WIND_THRESHOLD_MS) &
    (df_master["max_swh_m"]   <= WAVE_THRESHOLD_M)
).astype(int)

print("\nSelected Thresholds:")
print(f"Wind threshold = {WIND_THRESHOLD_MS:.2f} m/s ({WIND_THRESHOLD_MS * MS_TO_KNOTS:.2f} k
print(f"Wave threshold = {WAVE_THRESHOLD_M:.2f} m")
```

```
Selected Thresholds:
Wind threshold = 8.50 m/s (16.52 knots)
Wave threshold = 1.75 m
```

I REMOVE FEB 29 FOR CALENDAR ALIGNMENT

```
df_prob = df_master.copy()
df_prob["date"] = pd.to_datetime(df_prob["date"])

df_prob = df_prob[
    ~((df_prob["date"].dt.month == 2) & (df_prob["date"].dt.day == 29))
].copy()

df_prob["month_day"] = df_prob["date"].dt.strftime("%m-%d")

calendar_365 = pd.date_range("2001-01-01", "2001-12-31", freq="D")
calendar_df = pd.DataFrame({
    "contract_date_dummy": calendar_365,
    "month_day": calendar_365.strftime("%m-%d"),
    "day_of_year": np.arange(1, 366)
})
```

J CONTRACT WINDOW

```
start_dummy = pd.to_datetime(f"2001-{CONTRACT_START_MMDD}")
end_dummy   = pd.to_datetime(f"2001-{CONTRACT_END_MMDD}")

if start_dummy <= end_dummy:
    contract_calendar = calendar_df[
        (calendar_df["contract_date_dummy"] >= start_dummy) &
        (calendar_df["contract_date_dummy"] <= end_dummy)
    ].copy()
else:
    contract_calendar = calendar_df[
        (calendar_df["contract_date_dummy"] >= start_dummy) |
        (calendar_df["contract_date_dummy"] <= end_dummy)
    ].copy()

contract_calendar = contract_calendar.reset_index(drop=True)
contract_calendar["contract_day_index"] = np.arange(1, len(contract_calendar) + 1)

contract_days = len(contract_calendar)
contract_years = contract_days / 365.0

print("\nContract Window:")
print(f"Start (MM-DD): {CONTRACT_START_MMDD}")
print(f"End   (MM-DD): {CONTRACT_END_MMDD}")
print(f"Contract length: {contract_days} days")
print(f"Contract length: {contract_years:.6f} years")
```

```
Contract Window:
Start (MM-DD): 01-01
End   (MM-DD): 12-31
Contract length: 365 days
Contract length: 1.000000 years
```

K DAILY FISHABLE PROBABILITY FOR CONTRACT DAYS ONLY

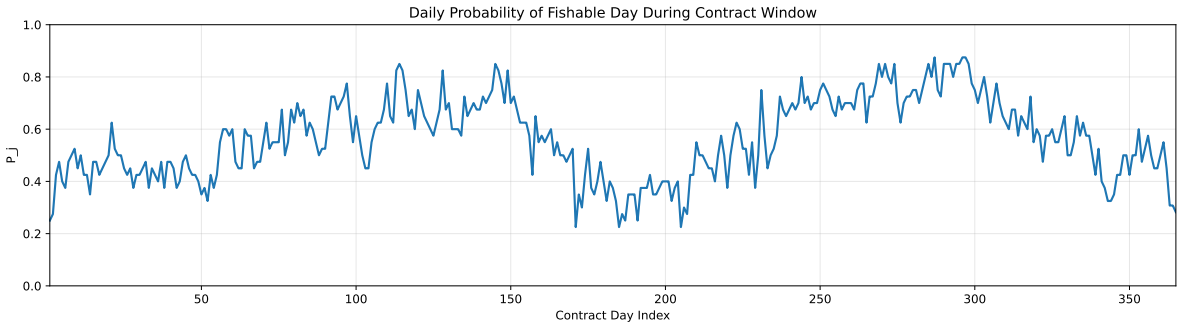
```
daily_prob_all = (
    df_prob.groupby("month_day")
    .agg(
        p_fishable=("fdi", "mean"),
        fishable_count=("fdi", "sum"),
        n_years=("fdi", "count")
    )
    .reset_index()
)

daily_prob_contract = contract_calendar.merge(
    daily_prob_all,
    on="month_day",
    how="left"
)

if daily_prob_contract["p_fishable"].isna().any():
    raise ValueError("Some contract dates have missing fishable probabilities.")

expected_fishable_days_contract = daily_prob_contract["p_fishable"].sum()

print("\nContract Probability Summary:")
print(f"Expected fishable days during contract = {expected_fishable_days_contract:.2f}")

# CONTRACT-YEAR FDI FOR HISTORICAL YEARS

contract_month_days = set(contract_calendar["month_day"])

df_contract_only = df_prob[df_prob["month_day"].isin(contract_month_days)].copy()

annual_contract_fdi = (
    df_contract_only.groupby("year")
```

```
    .agg(
        contract_fdi=("fdi", "sum"),
        contract_days=("fdi", "count"),
        pct_fishable=("fdi", lambda x: x.mean() * 100)
    )
    .reset_index()
)

mean_contract_fdi = annual_contract_fdi["contract_fdi"].mean()
std_contract_fdi  = annual_contract_fdi["contract_fdi"].std()

print("\nContract FDI Summary:")
print(f"Mean contract FDI = {mean_contract_fdi:.2f} days")
print(f"Std contract FDI  = {std_contract_fdi:.2f} days")
print(f"Average profit/day = ${AVG_PROFIT_PER_DAY:,.2f}")
```

```
Contract Probability Summary:
Expected fishable days during contract = 206.80

Contract FDI Summary:
Mean contract FDI = 206.78 days
Std contract FDI  = 23.94 days
Average profit/day = $330.00
```

L PLOT DAILY FISHABLE PROBABILITY

```
plt.figure(figsize=(14, 4))
plt.plot(
    daily_prob_contract["contract_day_index"],
    daily_prob_contract["p_fishable"],
    linewidth=1.8
)
plt.title("Daily Probability of Fishable Day During Contract Window")
plt.xlabel("Contract Day Index")
plt.ylabel("P_j")
plt.xlim(1, contract_days)
plt.ylim(0, 1)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



M POISSON-BINOMIAL PMF

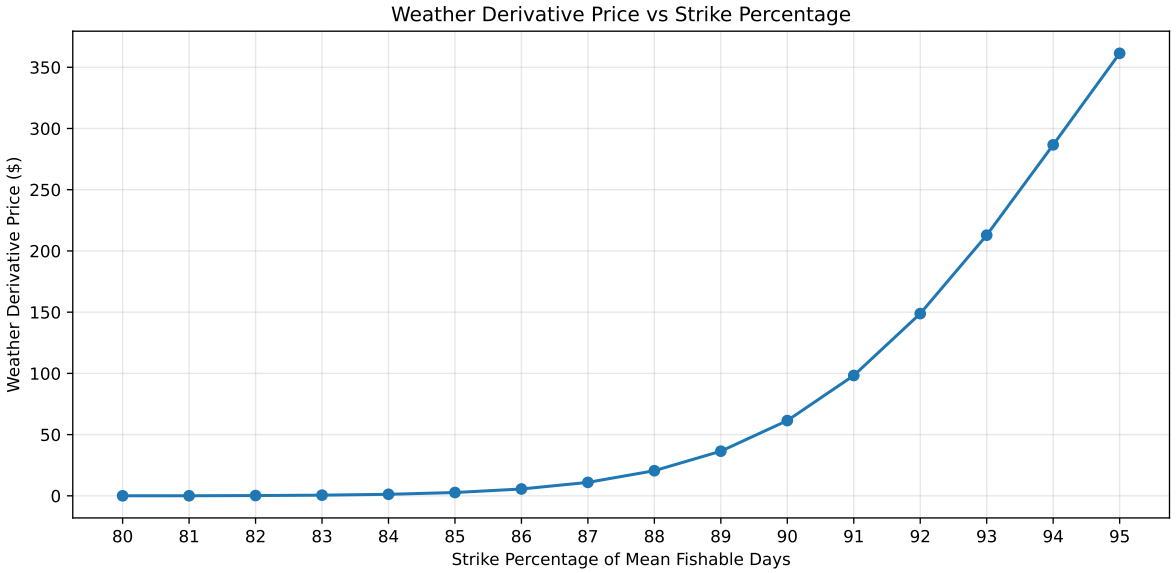
N ==========

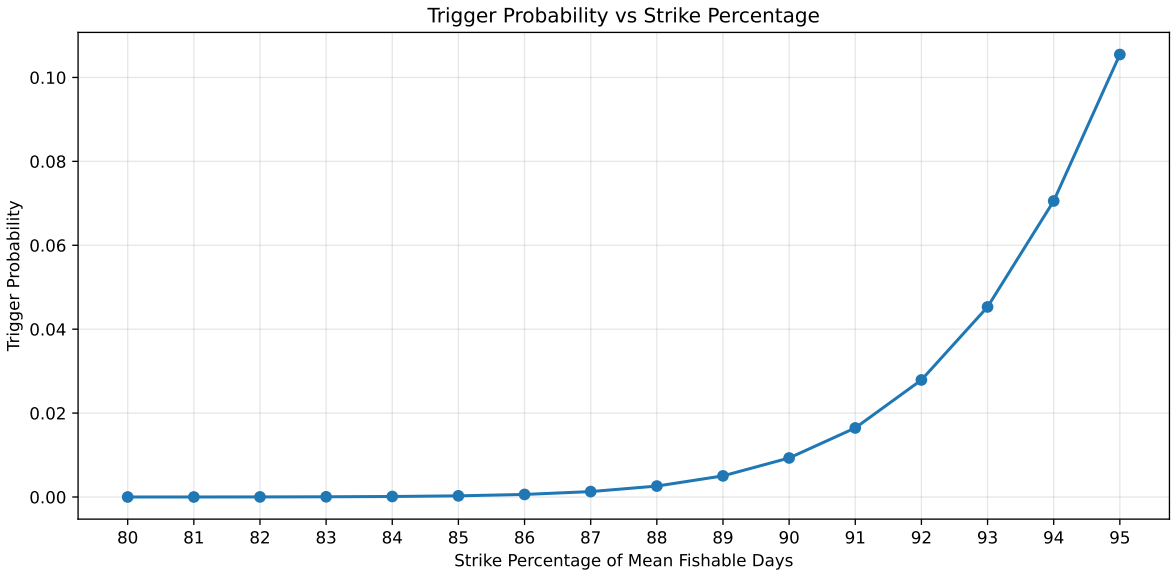
```
p = daily_prob_contract["p_fishable"].to_numpy()
n = len(p)

pmf = np.zeros(n + 1)
pmf[0] = 1.0

for pj in p:
    new_pmf = np.zeros(n + 1)
    new_pmf[0] = pmf[0] * (1 - pj)
    for k in range(1, n + 1):
        new_pmf[k] = pmf[k] * (1 - pj) + pmf[k - 1] * pj
    pmf = new_pmf

discount_factor = np.exp(-RISK_FREE_RATE * contract_years)

# MULTI-STRIKE PRICING TABLE || Trigger rule: payout when N < K

pricing_rows = []

for strike_percent in STRIKE_PERCENTS:
    strike_days = int(round(mean_contract_fdi * strike_percent))

    # Fixed payout size based on strike shortfall from mean
    loss_amount = max(mean_contract_fdi - strike_days, 0) * AVG_PROFIT_PER_DAY

    # Correct trigger rule: fishable days strictly below strike
    # P(N < K) = sum from 0 to K-1
    if strike_days <= 0:
        trigger_probability = 0.0
    else:
        trigger_probability = pmf[:strike_days].sum()

    weather_derivative_price = discount_factor * loss_amount * trigger_probability
```

```
    pricing_rows.append({
        "strike_percent": strike_percent * 100,
        "strike_days": strike_days,
        "fixed_payout_L": loss_amount,
        "trigger_probability_P_N_less_K": trigger_probability,
        "weather_derivative_price": weather_derivative_price
    })

pricing_table = pd.DataFrame(pricing_rows)

print("\nPricing Table for Strike Percent 80% to 95%:")
print(
    pricing_table.round({
        "strike_percent": 0,
        "strike_days": 0,
        "fixed_payout_L": 2,
        "trigger_probability_P_N_less_K": 6,
        "weather_derivative_price": 2
    }).to_string(index=False)
)



# PLOT - STRIKE % VS PRICE

plt.figure(figsize=(10, 5))
plt.plot(
    pricing_table["strike_percent"],
    pricing_table["weather_derivative_price"],
    marker="o",
    linewidth=1.8
)
plt.title("Weather Derivative Price vs Strike Percentage")
plt.xlabel("Strike Percentage of Mean Fishable Days")
plt.ylabel("Weather Derivative Price ($)")

plt.xticks(STRIKE_PERCENTS * 100)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
# PLOT - STRIKE % VS TRIGGER PROBABILITY

plt.figure(figsize=(10, 5))
plt.plot(
    pricing_table["strike_percent"],
    pricing_table["trigger_probability_P_N_less_K"],
    marker="o",
    linewidth=1.8
)
plt.title("Trigger Probability vs Strike Percentage")
plt.xlabel("Strike Percentage of Mean Fishable Days")
plt.ylabel("Trigger Probability")

plt.xticks(STRIKE_PERCENTS * 100)

plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
Pricing Table for Strike Percent 80% to 95%:
 strike_percent  strike_days  fixed_payout_L  trigger_probability_P_N_less_K  weather_deriva
           80.0          165        13785.75                        0.000002
           81.0          167        13125.75                        0.000004
           82.0          170        12135.75                        0.000019
           83.0          172        11475.75                        0.000049
           84.0          174        10815.75                        0.000119
           85.0          176        10155.75                        0.000276
           86.0          178         9495.75                        0.000610
           87.0          180         8835.75                        0.001289
           88.0          182         8175.75                        0.002605
           89.0          184         7515.75                        0.005032
           90.0          186         6855.75                        0.009303
           91.0          188         6195.75                        0.016460
           92.0          190         5535.75                        0.027895
           93.0          192         4875.75                        0.045302
           94.0          194         4215.75                        0.070555
           95.0          196         3555.75                        0.105465
```


Weather Derivative Price vs Strike Percentage
Weather Derivative Price ($)
0
50
100
150
200
250
300
350
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
Strike Percentage of Mean Fishable Days


Trigger Probability vs Strike Percentage
Trigger Probability
0.00
0.02
0.04
0.06
0.08
0.10
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
Strike Percentage of Mean Fishable Days